Introduction to Embedded Systems

Course Code: 244066

Chapter 3: General IO ports of STM32

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

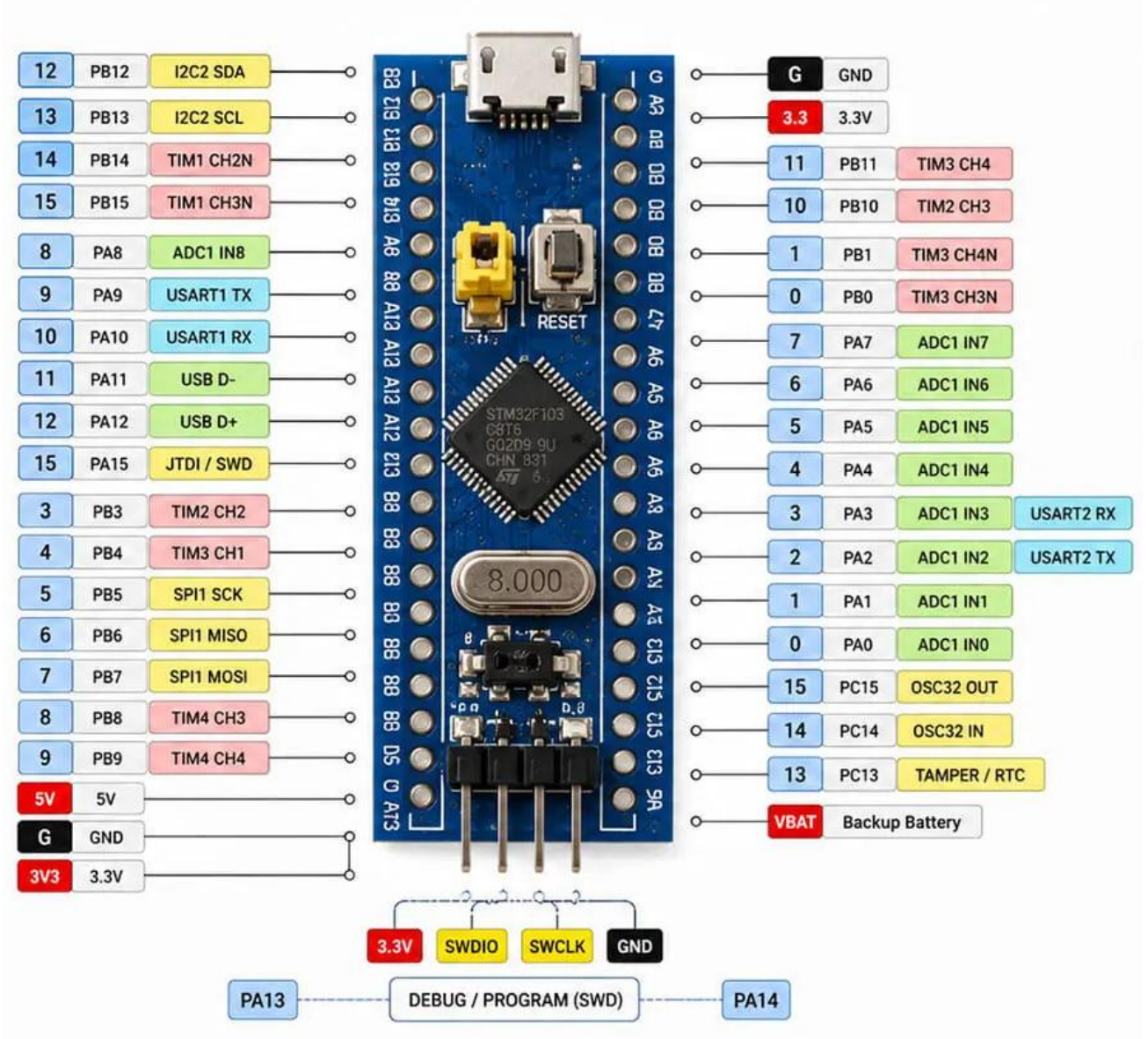
Contents

1. **Introduction to GPIO**
2. GPIO Features in STM32
   - Configurable Modes
   - Configurable Speed
   - Internal Pull-up/Pull-down Resistors
3. GPIO Registers in STM32
   - Important Registers
   - Steps to configure GPIO
4. Simulated Applications
   - Example 1: Flowing Light (Output Mode)
   - Example 2: Input Mode (Key Detection)
5. Keil/MDK-ARM Project Setup
6. Common Mistakes in Development
7. Summary

3.1. Introduction to GPIO

GPIO = ***G**eneral **P**urpose **I**nput/**O**utput*.

- It provides a flexible digital signal interface between the microcontroller and external devices.



3.1. Introduction to GPIO

Role in Embedded Systems:

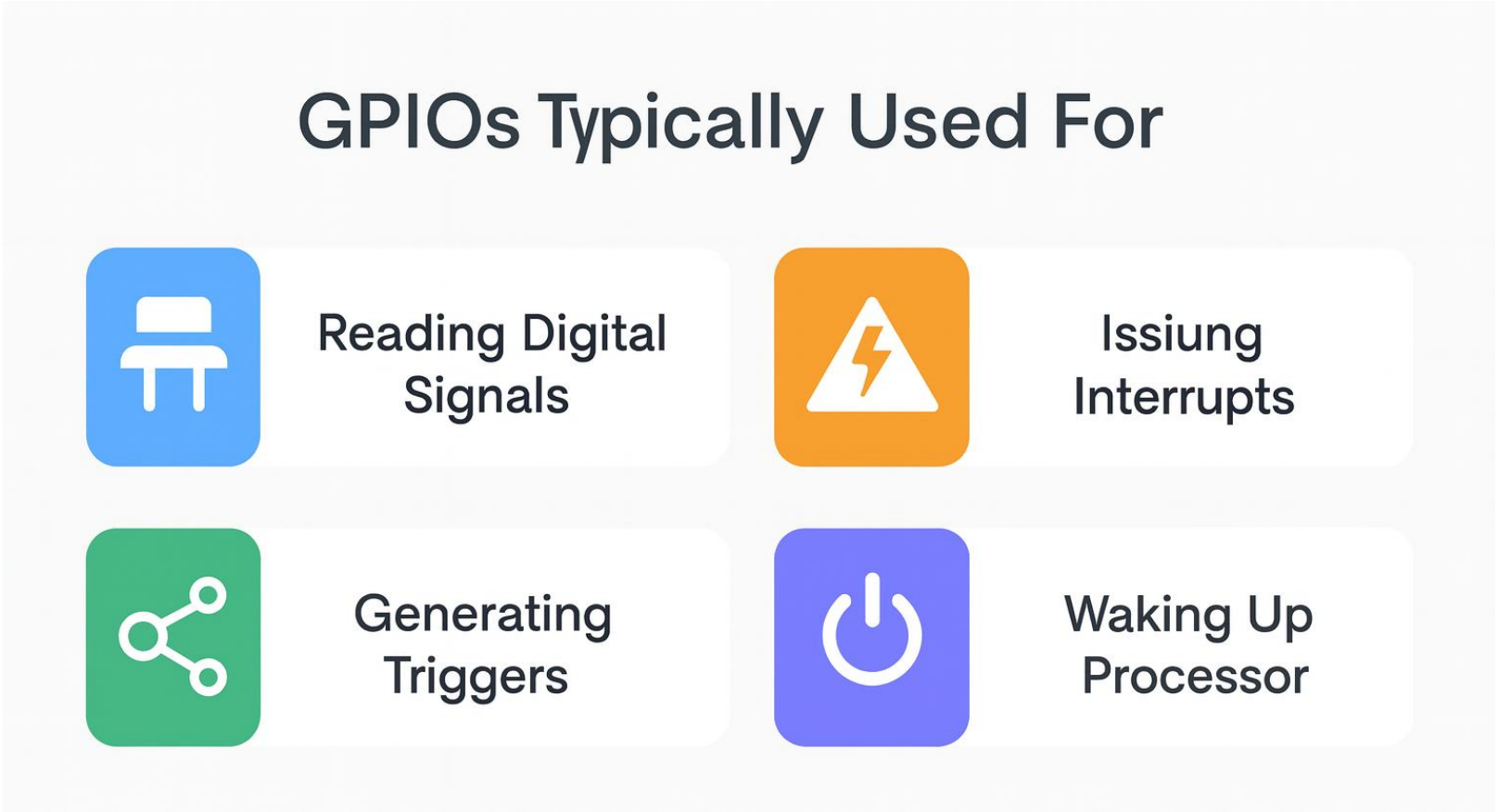
- Foundation of all hardware interaction.
- Input → Read signals (e.g., buttons, sensors).
- Output → Control devices (e.g., LEDs, motors, buzzers).

Examples:

- Smartphone buttons → GPIO inputs.
- LED indicators → GPIO outputs.
- Keypads, relays, touch sensors → GPIO applications.



3.1. Introduction to GPIO



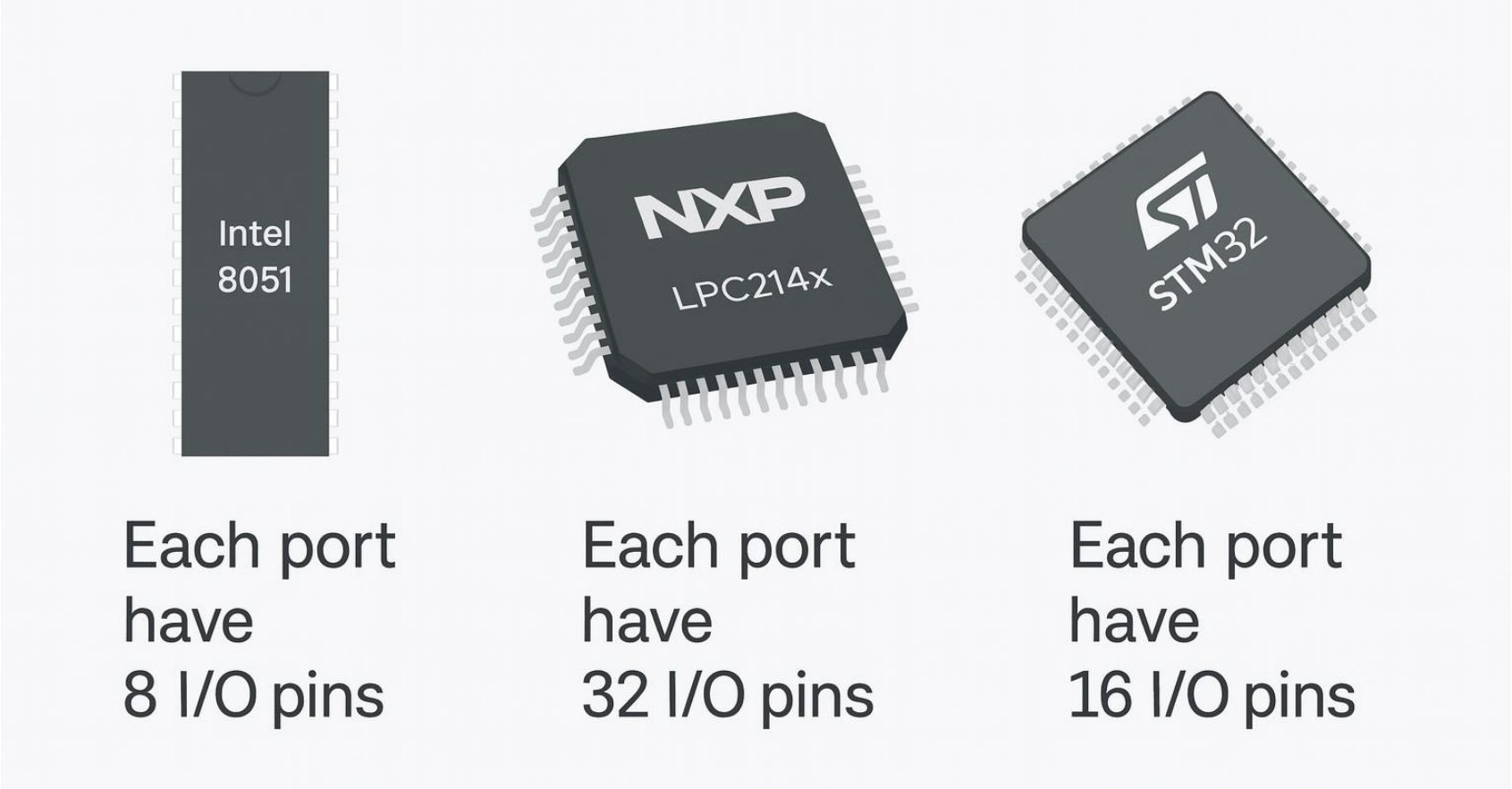
Contents

1. Introduction to GPIO
2. **GPIO Features in STM32**
   - Configurable Modes
   - Configurable Speed
   - Internal Pull-up/Pull-down Resistors
3. GPIO Registers in STM32
   - Important Registers
   - Steps to configure GPIO
4. Simulated Applications
   - Example 1: Flowing Light (Output Mode)
   - Example 2: Input Mode (Key Detection)
5. Keil/MDK-ARM Project Template
6. Common Mistakes in Development
7. Summary

3.2. GPIO Features in STM32



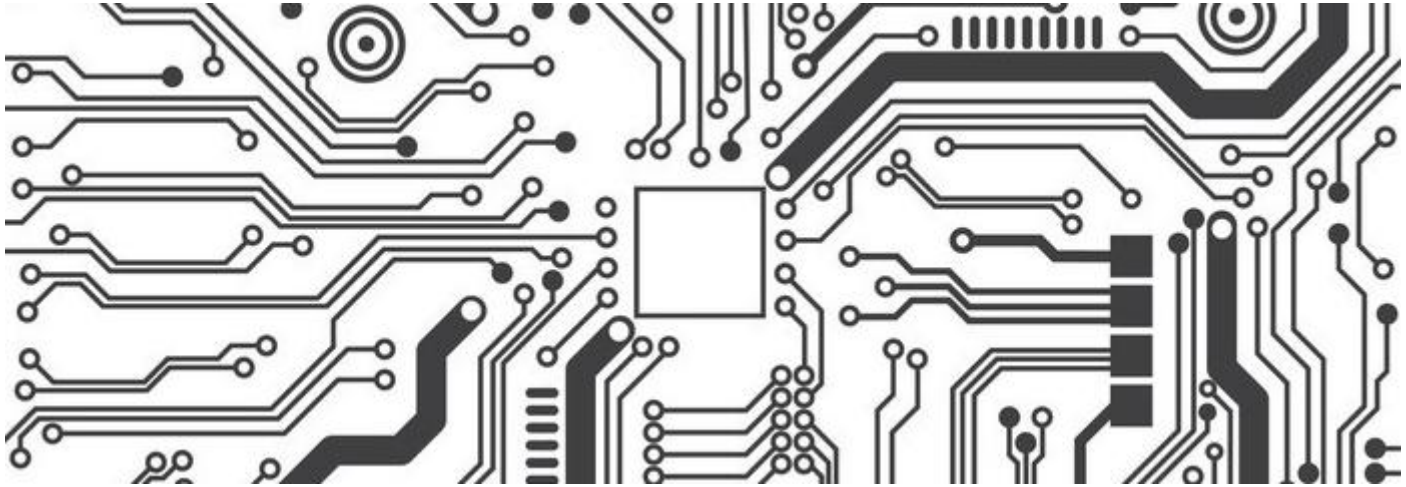
3.2. GPIO Features in STM32



3.2. GPIO Features in STM32

Pull Resistors

- A **resistor** can connect the pin to either **Vcc (logic 1)** or **GND (logic 0)** to keep it stable when no external signal is present.
  - **Pull-up resistor** → Keeps the pin HIGH when not pressed.
  - **Pull-down resistor** → Keeps the pin LOW when not pressed.



3.2. GPIO Features in STM32

- **Why Do We Need Them?**
  - A **GPIO input pin** must always be in a **defined state** (either 0 or 1).
  - If left unconnected, the pin "floats" → it may pick up noise and randomly switch between HIGH and LOW.
  - This leads to **unreliable readings** (false button presses, unstable sensor values).

3.2. GPIO Features in STM32

Internal Pull-up / Pull-down in STM32

- STM32 provides **built-in resistors** that can be enabled in software
- no need for external components.
- This saves **hardware space, cost, and wiring complexity**.

3.2. GPIO Features in STM32

- It’s like a **light switch spring**:
  - Without a spring, the switch may get stuck halfway (undefined).
  - With a spring (pull-up/pull-down), the switch always rests in a clear ON or OFF state when untouched.



- Internal pull-up/pull-down resistors ensure that input pins **don’t float** and always give reliable HIGH/LOW signals.

3.2. GPIO Features in STM32

Multiplexed Pins – One Pin, Many Roles.

- In STM32, **one physical pin** can perform **different functions** depending on how we configure it.
- Example: Pin **PA0**
  - It can work as a **digital input** (read button press).
  - Or as a **digital output** (control an LED).
  - Or as **alternate function** (used by USART or timer).
  - Or as **analog input to** internal peripherals (for ADC).

3.2. GPIO Features in STM32

- It’s like a **Swiss Army Knife**: one tool (the pin) can act as a knife, screwdriver, or bottle opener, depending on how you unfold it.



3.2. GPIO Features in STM32

Configurable Modes

Input Mode



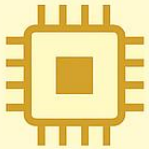
- Floating (no pull resistor)
- Pull-up or Pull-down

Output Mode



- Push-pull
- Open-drain

Alternate Functiion Mode



- Pin connected internally to peripherals like USART, SPI, I²C

Analog Mode



- Used when pin is connected to ADC/DAC

3.2. GPIO Features in STM32

Input Mode

Used for the microcontroller to **receive a signal**.

Types of Input:

- **Floating Input**
  - Pin is left unconnected.
  - Can pick up random noise (unreliable).
  - Rarely used without care.

3.2. GPIO Features in STM32

- **Example**: Like a microphone left open in a noisy room.



3.2. GPIO Features in STM32

- **Pull-up / Pull-down Input**
  - Internal resistors connect the pin to **Vcc (pull-up)** or **GND (pull-down)**.
  - Keeps the signal stable when no button/sensor is pressed.
  - **Example**:
    - Pull-Down: Motion detectors.
    - Pull-Up: Magnetic door sensors.

3.2. GPIO Features in STM32

Output Mode

Used when the microcontroller needs to **send a signal** (turn something ON/OFF).

Types of Output:

- **Push-Pull**
  - Pin can drive both **HIGH (1)** and **LOW (0)** strongly.
  - Common for LEDs, buzzers, relays.

3.2. GPIO Features in STM32

- **Open-Drain**
  - A Pin can only pull LOW; to go HIGH it needs an **external resistor**.
  - Common for **I²C bus** where multiple devices share the line.

- **Analogy**: A group project where everyone can "pull the rope down," but if nobody pulls, the rope naturally stays up because of a spring (resistor).

```
GPIOC->OTYPER |= (1 << 13); // PC13 open-drain output
```

3.2. GPIO Features in STM32

Alternate Function Mode

- Pin is **internally connected** to another module inside STM32.
- Used when the pin is not just simple input/output but part of a communication system.
- Examples:
  - **USART TX/RX** → for serial communication.
  - **SPI MOSI/MISO** → for high-speed data transfer.
  - **PWM (Timer Output)** → for motor speed control.

3.2. GPIO Features in STM32

Analog Mode

- Pin is connected to **ADC (**Analog to Digital Converter**)** or **DAC (**Digital to Analog Converter**).**
- Used when we need to measure real-world signals like:
  - Temperature (sensor voltage).
  - Microphone audio.
  - Light sensor brightness.



- **Analogy**: Instead of just **YES/NO (digital)**, the pin can now understand **gradual values** (like a dimmer switch instead of a simple ON/OFF switch).

3.2. GPIO Features in STM32

Configurable Speed of GPIO

What is "GPIO Speed"?

- When we configure a pin as an **output**, it can switch between HIGH (1) and LOW (0).
- **Configurable speed** means how **fast** the pin can change its state.
- In STM32, typical options are: **2 MHz, 25 MHz, 50 MHz, 100 MHz**.

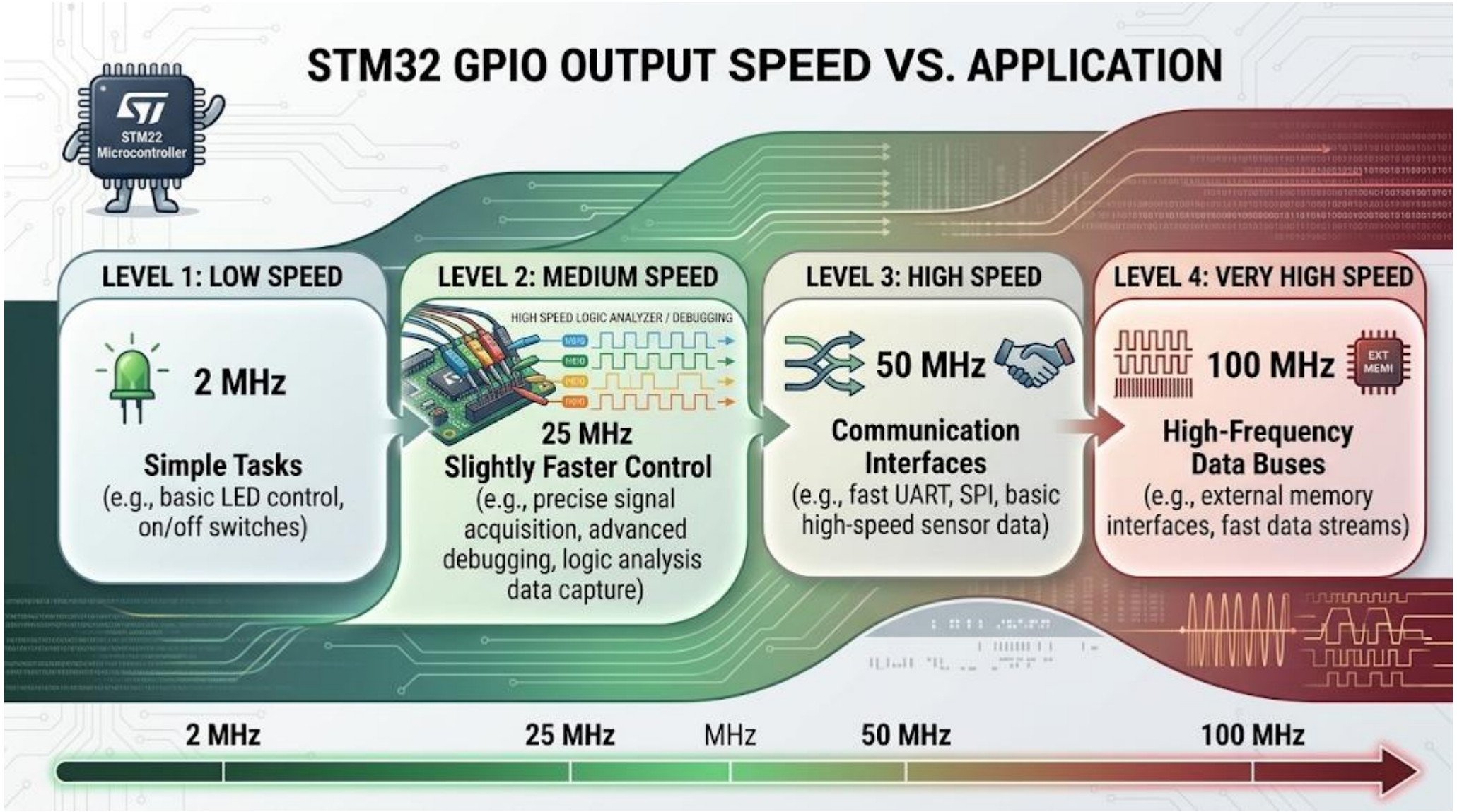
3.2. GPIO Features in STM32

Why Do We Need Different Speeds?

- Not every application needs very fast signals.
- Using a higher speed:
  - Produces **sharper, faster edges**.
  - But also consumes **more power** and causes **electrical noise**.
- Using a lower speed:
  - **Saves** power.
  - **Reduces** interference.
  - Is **enough** for slow signals like LEDs.



3.2. GPIO Features in STM32



3.2. GPIO Features in STM32

Choosing the right speed is about **balance**:

- Too low → system may be too slow.
- Too high → wastes power or causes signal integrity problems.

- GPIO speed is not about how *fast the LED blinks to your eye*, but how quickly the **electrical signal can switch**.
- Always pick the **lowest** speed that is still enough for your application.

Contents

1. Introduction to GPIO
2. GPIO Features in STM32
   - Configurable Modes
   - Configurable Speed
   - Internal Pull-up/Pull-down Resistors
3. **GPIO Registers in STM32**
   - Important Registers
   - Steps to configure GPIO
4. Simulated Applications
   - Example 1: Flowing Light (Output Mode)
   - Example 2: Input Mode (Key Detection)
5. Keil/MDK-ARM Project Template
6. Common Mistakes in Development
7. Summary

3.3. GPIO Registers in STM32

What are Registers?

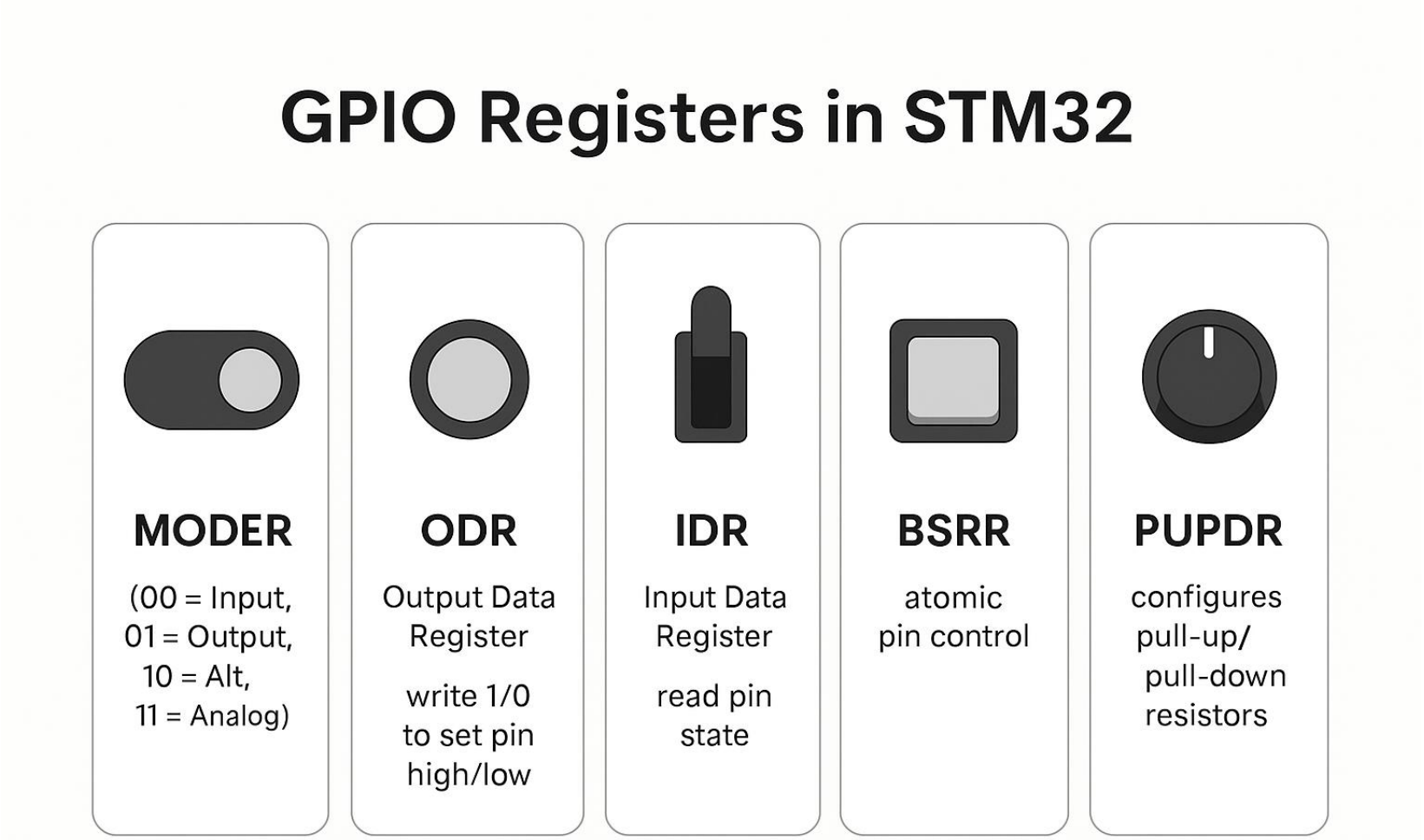
- Each GPIO pin is controlled through **special memory locations** called **registers**.
- **Writing** or **reading** from these registers changes how the pin behaves.







3.3. GPIO Registers in STM32



3.3. GPIO Registers in STM32

Important GPIO Registers

MODER (Mode Register)

- Defines how each pin will behave.
- Each pin takes **2 bits** in this register:
  - 00 → Input
  - 01 → Output
  - 10 → Alternate Function (UART, SPI, etc.)
  - 11 → Analog

3.3. GPIO Registers in STM32

MODER (Mode Register)

- **Example:**

```
GPIOA->MODER |= (1 << 0); // Set PA0 as Output
```

- **Analogy:** Like setting the "**job**" of a worker (input, output, or specialist role).

3.3. GPIO Registers in STM32

ODR (Output Data Register)

- Stores the output values for all pins.
- Writing 1 → sets pin HIGH (3.3V).
- Writing 0 → sets pin LOW (0V).

- **Example:**

```
GPIOC->ODR |= (1 << 13);  // Turn LED ON at PC13
GPIOC->ODR &= ~(1 << 13); // Turn LED OFF
```

- **Analogy:** Like flipping a switch to ON/OFF for each pin.

3.3. GPIO Registers in STM32

IDR (Input Data Register)

- Shows the current state of input pins.
- Read 1 → pin is HIGH.
- Read 0 → pin is LOW.

- **Example:**

```
if (GPIOA->IDR & (1 << 0)) {
    // Button pressed
}
```

- **Analogy:** Like reading whether a doorbell button is pressed (1) or not (0).

3.3. GPIO Registers in STM32

BSRR (Bit Set/Reset Register)

- Special register for **atomic control** (set or reset a pin in one instruction).
- Writing to **lower 16 bits** → Sets the corresponding pin.
- Writing to **upper 16 bits** → Resets the corresponding pin.

- **Example:**

```
GPIOC->BSRR = (1 << 13);         // Set PC13 HIGH
GPIOC->BSRR = (1 << (13 + 16)); // Reset PC13 LOW
```

- **Why atomic?** Prevents errors when multiple tasks access pins at the same time.
- **Analogy:** Like pressing a button that instantly locks/unlocks a door without delay.

3.3. GPIO Registers in STM32

PUPDR (Pull-up/Pull-down Register)

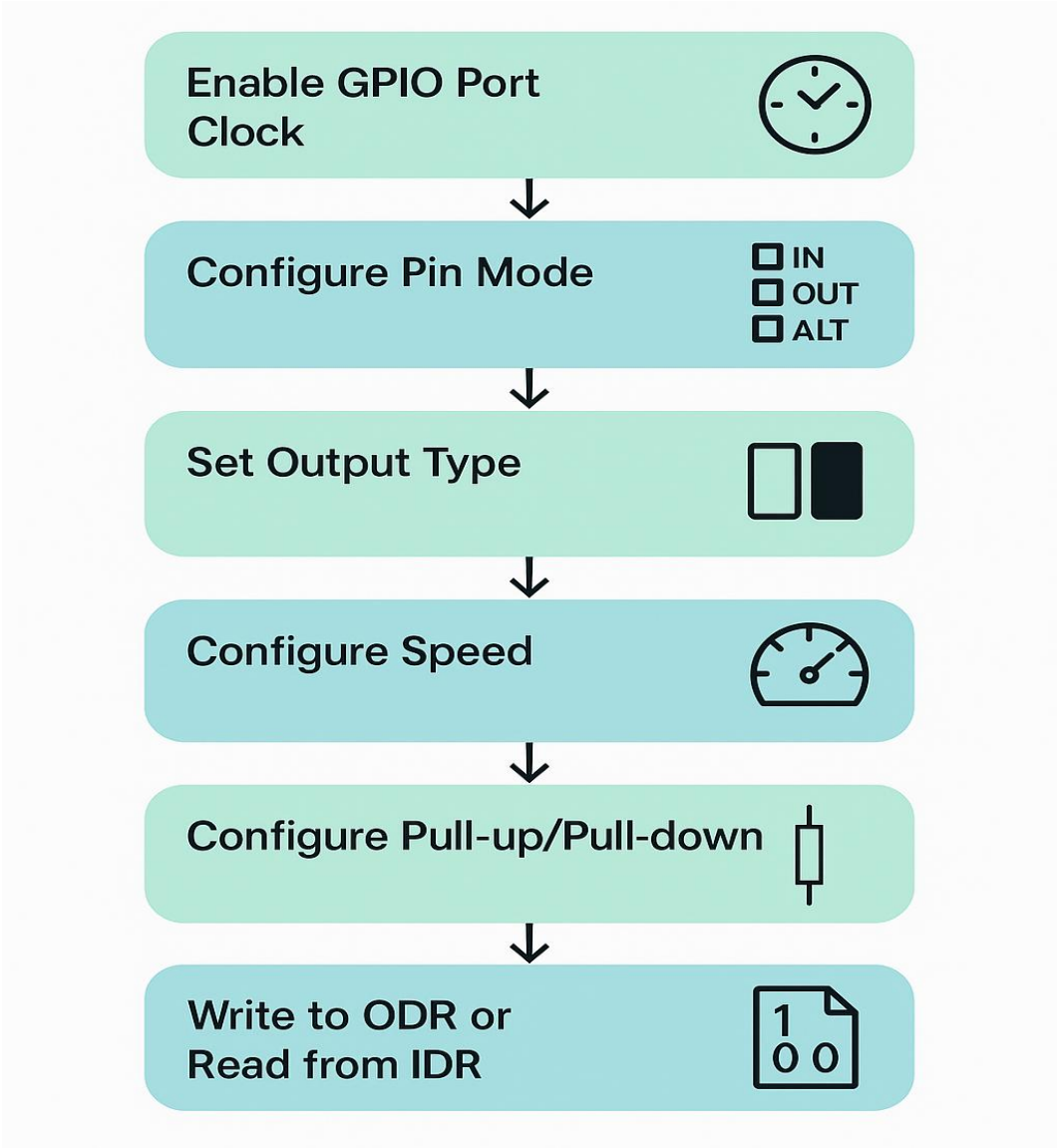
- Configures whether the pin uses **internal pull-up or pull-down resistors**.
- Each pin uses 2 bits:
  - 00 → No pull (floating).
  - 01 → Pull-up.
  - 10 → Pull-down.
- **Example:**

```
GPIOA->PUPDR |= (1 << 0); // Enable Pull-up on PA0
```

- **Analogy:** Like adding a spring to a switch to make sure it rests in a default position.

3.3. GPIO Registers in STM32

Configuration of GPIO in STM32



3.3. GPIO Registers in STM32

Step 1: Enable GPIO Port Clock (RCC_AHB1ENR)

- Before using a GPIO pin, you must **turn on its power supply (clock)**.
- Each GPIO port (A, B, C…) has its own clock.
- This is done in the **RCC_AHB1ENR register**.

- **Example:**

```
RCC->AHB1ENR |= (1 << 0);  // Enable clock for GPIOA
```

- **Analogy:** Like switching on the main power before using a room's appliances.

3.3. GPIO Registers in STM32

Step 2: Configure Pin Mode (MODER)

- Set the pin function: Input, Output, Alternate, or Analog.
- Each pin takes **2 bits** in the MODER register.

- **Example:**

```
GPIOA->MODER |= (1 << (2*5));  // Set PA5 as output
```

- **Analogy:** Assigning a job role to a worker (input, output, specialist, or analog sensor).

3.3. GPIO Registers in STM32

Step 3: Set Output Type (OTYPER)

- If pin is in **Output mode**, choose:
  - **Push-pull** → strong drive both HIGH and LOW (default, for LEDs).
  - **Open-drain** → can only pull LOW (needs external pull-up, for I²C).

- **Example:**

```
GPIOA->OTYPER &= ~(1 << 5);  // PA5 as push-pull
```

- **Analogy:** Push-pull is like a strong hand pushing both ways,
  open-drain is like only pulling down with gravity.

3.3. GPIO Registers in STM32

Step 4: Configure Speed (OSPEEDR)

- Defines how fast the pin can switch: 2, 25, 50, 100 MHz.
- Use **low speed for LEDs**, **high speed for communication pins**.

- **Example:**

```
GPIOA->OSPEEDR |= (2 << (2*5)); // Medium speed for PA5
```

- **Analogy:** Like choosing the speed limit of a road, no need for a highway when walking to class!

3.3. GPIO Registers in STM32

Step 5: Configure Pull-up/Pull-down (PUPDR)

- Prevents input pins from "floating" (undefined states).
- Options:
  - 00 → No pull (floating).
  - 01 → Pull-up.
  - 10 → Pull-down.

- **Example**:

```
GPIOA->PUPDR |= (1 << (2*0)); // Enable pull-up on PA0
```

- **Analogy:** Like a spring in a switch → ensures it rests in a default position.

3.3. GPIO Registers in STM32

Step 6: Write to ODR or Read from IDR

- **Output Data Register (ODR):**
  - Write 1 to set pin HIGH, 0 to set pin LOW.
  - **Example** → LED blinking.

```
GPIOA->ODR |= (1 << 5);  // Set PA5 HIGH
GPIOA->ODR &= ~(1 << 5); // Set PA5 LOW
```

- **Input Data Register (IDR):**
  - Read the current pin value (0 = LOW, 1 = HIGH).
  - **Example** → Button detection.

```
if (GPIOA->IDR & (1 << 0)) { /* Button pressed */ }
```

Analogy:

- **ODR** = Light switch (you turn it ON/OFF).
- **IDR** = Sensor reading (you check if the light is ON/OFF).

Contents

1. Introduction to GPIO
2. GPIO Features in STM32
   - Configurable Modes
   - Configurable Speed
   - Internal Pull-up/Pull-down Resistors
3. GPIO Registers in STM32
   - Important Registers
   - Steps to configure GPIO
4. **Simulated Applications**
   - Example 1: Flowing Light (Output Mode)
   - Example 2: Input Mode (Key Detection)
5. Keil/MDK-ARM Project Template
6. Common Mistakes in Development
7. Summary

3.4. Simulated Applications

Example 1: Output Mode (Flowing Light)
Objective: Control LEDs sequentially (flowing pattern).

Clock Enable
- Without this, pins won't work (like plugging a device without power).

Pin Configuration
- GPIOC->CRL = 0x33333333; sets PC0–PC7 as **output push-pull**.
- Each pin can now drive LEDs.

Main Loop (Flowing Pattern)
- The **for-loop** shifts a 1 through the Output Data Register (ODR).

Delay Function
- Slows down the **switching** so the human eye can see each LED clearly.

```c
#include "stm32f10x.h"

int main(void) {
    // Enable GPIOC clock
    RCC->APB2ENR |= (1 << 4);
    // Configure PC0-PC7 as output
    GPIOC->CRL = 0x33333333;

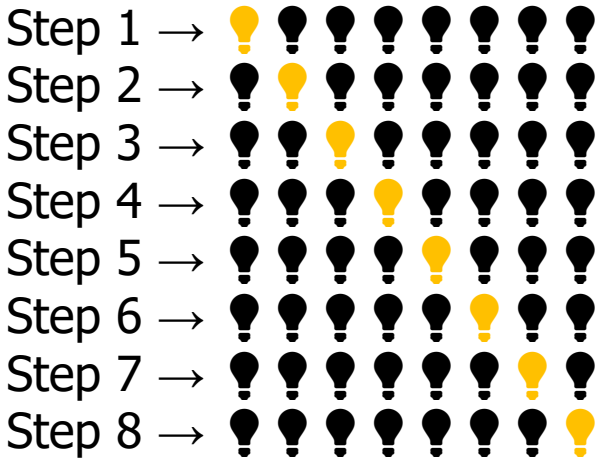
    while (1) {
        for (int i = 0; i < 8; i++) {
            GPIOC->ODR = (1 << i); // Turn on one LED at a time
            for (int j = 0; j < 100000; j++); // Delay
        }
    }
}
```

3.4. Simulated Applications

Example 1: Output Mode (Flowing Light)
Objective: Control LEDs sequentially (flowing pattern).

- Loop shifts a "1" across ODR bits.
- Each bit corresponds to an LED.
- Creates a flowing light effect.

Visual Explanation (LED Sequence)
Imagine 8 LEDs connected to PC0–PC7:



- **For Simulation:**
  Use debugger → Watch GPIOC->ODR → Observe bit shifting.

3.4. Simulated Applications

What is the Watch Window?

- The **Watch Window** in **MDK-ARM (Keil uVision)** allows us to monitor the **values of variables or registers in real time** while the program is running.
- It is like a "magnifying glass" into the microcontroller's memory.
- Instead of guessing what's happening, we can **see the actual value of registers** like GPIOx_ODR while debugging.

3.4. Simulated Applications

Why Use Watch Window for GPIO?

- The **Output Data Register (ODR)** controls which pins are HIGH (1) or LOW (0).
- By watching GPIOC->ODR, you can **see how the LED pattern changes in binary form** (e.g., 00000001, 00000010, 00000100 …).
- Even without physical LEDs, they can **understand the effect**.

3.4. Simulated Applications

Example 2: Input Mode (Key Detection)

Objective

- Learn how to use a **GPIO pin as input** to detect button presses.
- When the **button is pressed**, an **LED will turn ON**.
- This shows how microcontrollers **interact with the outside world**.

3.4. Simulated Applications

Example 2: Input Mode (Key Detection)
Objective: Detect button press and turn LED ON.

When button is NOT pressed:

- PA0 pulled HIGH by resistor.
- IDR reads 1.
- LED = OFF.

When button is PRESSED:

- PA0 connected to GND (LOW).
- IDR reads 0.
- LED = ON.

```c
int main(void) {
    // 1. Enable clocks for GPIOA (button) and GPIOC (LED)
    RCC->APB2ENR |= (1 << 2) | (1 << 4);

    // 2. Configure PA0 as input with pull-up
    GPIOA->CRL &= ~(0xF << 0);  // Clear PA0 config
    GPIOA->CRL |=  (0x4 << 0);  // Input with pull-up

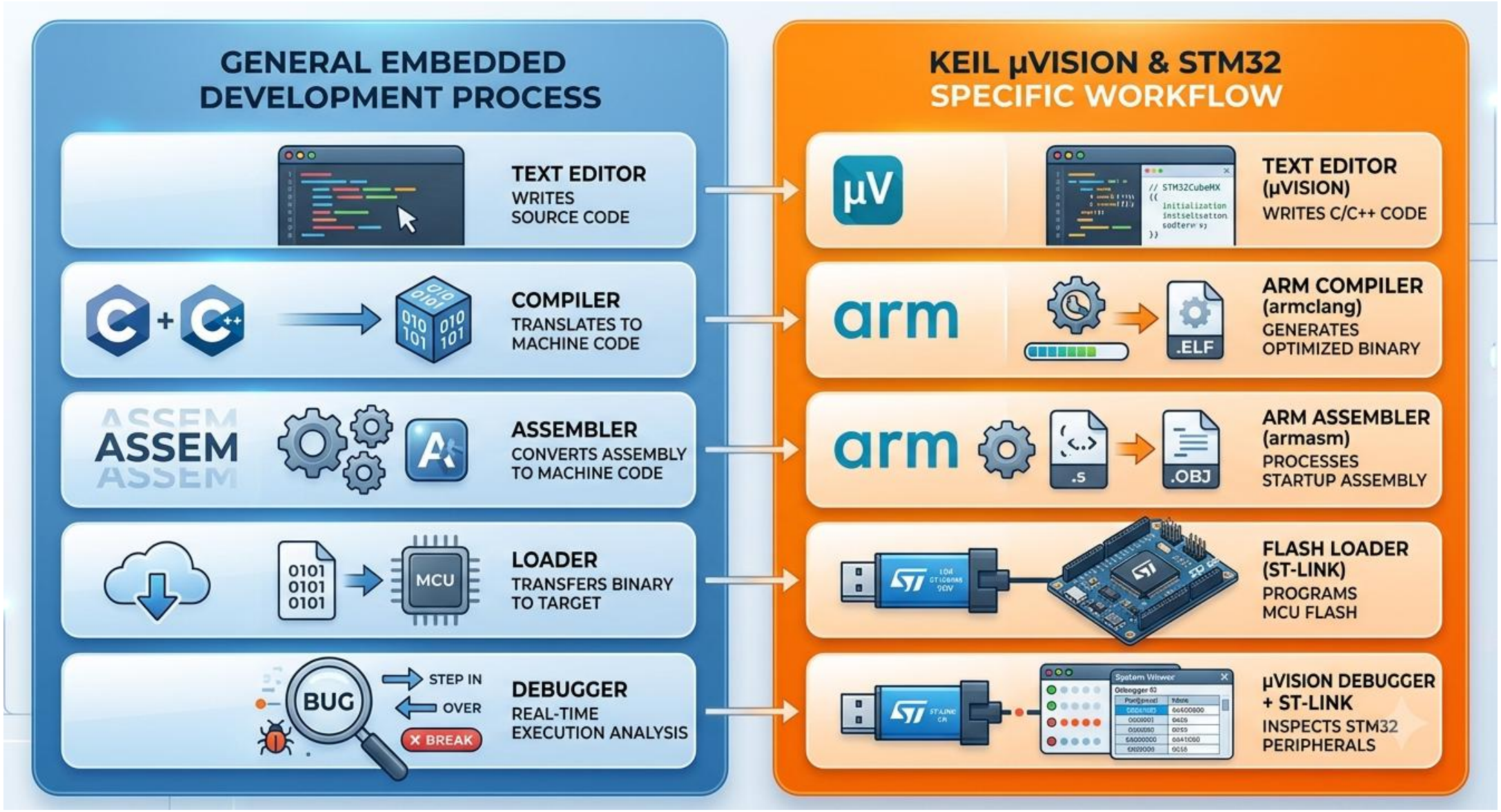
    // 3. Configure PC13 as output
    GPIOC->CRH &= ~(0xF << 20);
    GPIOC->CRH |=  (0x3 << 20); // Output push-pull

    while (1) {
        // 4. Read button state from IDR
        if ((GPIOA->IDR & 0x01) == 0) {
            // Button pressed (active low) → Turn LED ON
            GPIOC->ODR &= ~(1 << 13);
        } else {
            // Button not pressed → Turn LED OFF
            GPIOC->ODR |= (1 << 13);
        }
        Delay(10000); // Small debounce
    }
}
```

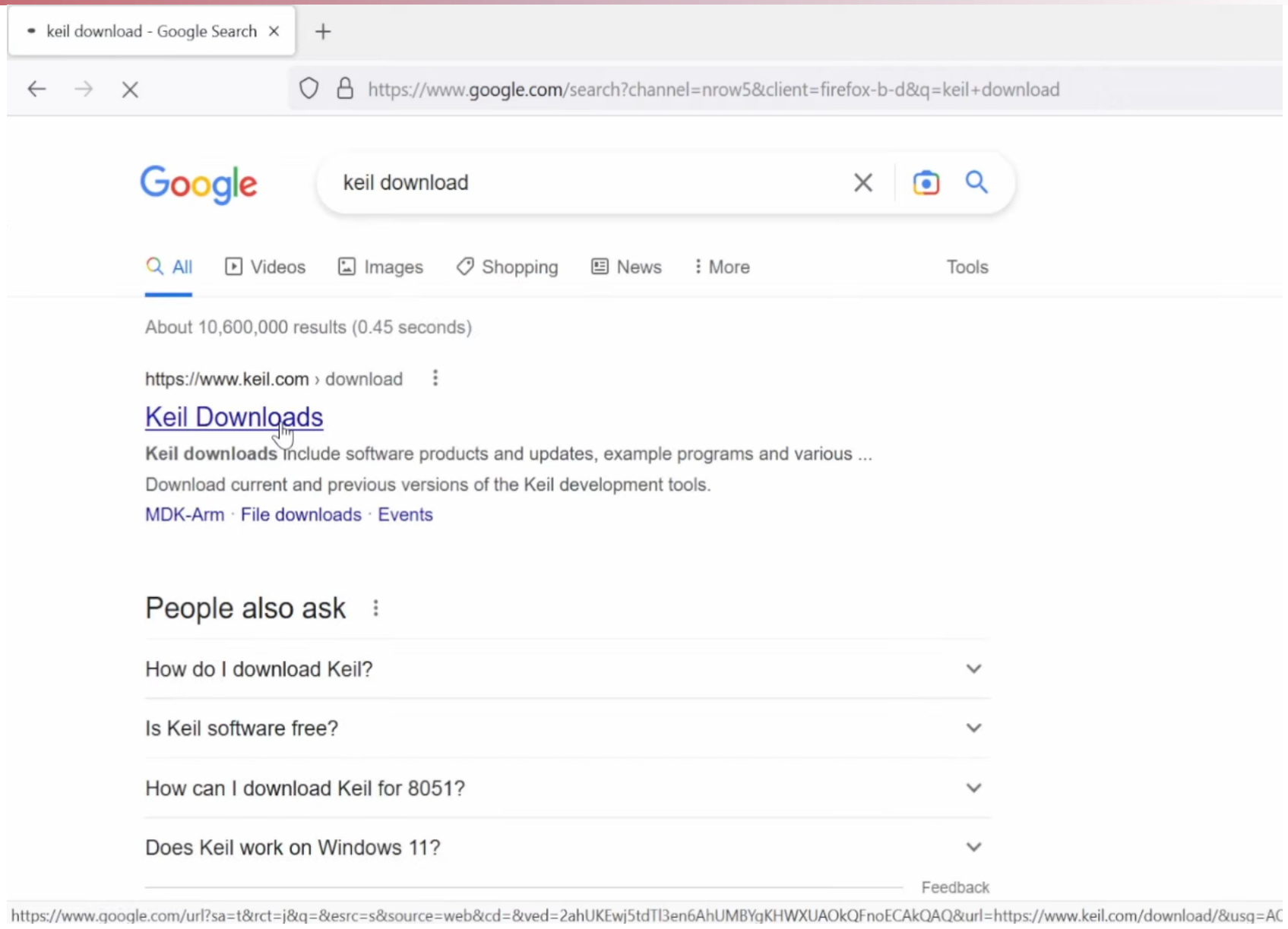
Contents

1. Introduction to GPIO
2. GPIO Features in STM32
   - Configurable Modes
   - Configurable Speed
   - Internal Pull-up/Pull-down Resistors
3. GPIO Registers in STM32
   - Important Registers
   - Steps to configure GPIO
4. Simulated Applications
   - Example 1: Flowing Light (Output Mode)
   - Example 2: Input Mode (Key Detection)
5. **Keil/MDK-ARM Project Setup**
6. Common Mistakes in Development
7. Summary

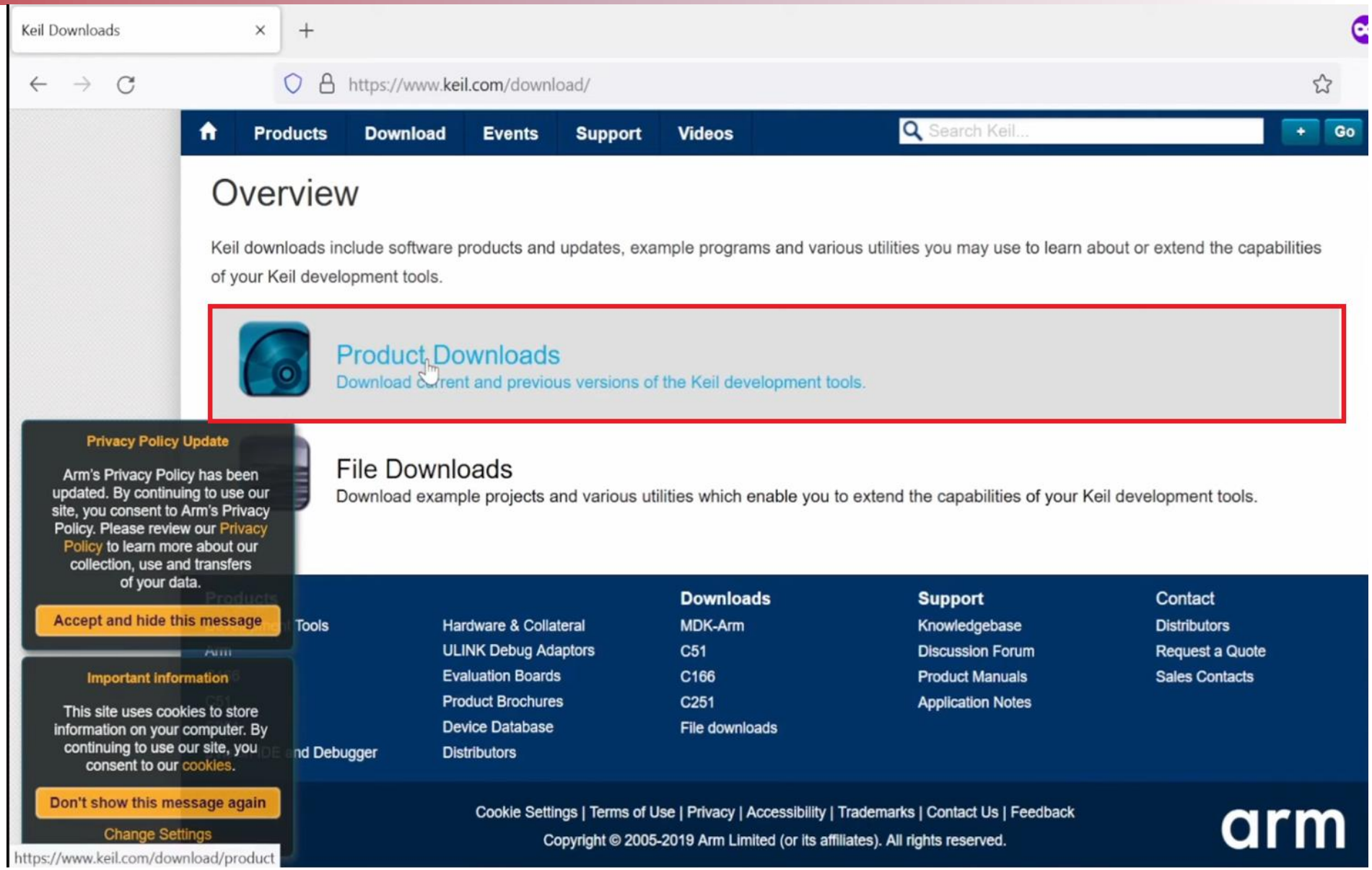
3.5. Keil/MDK-ARM Project Template



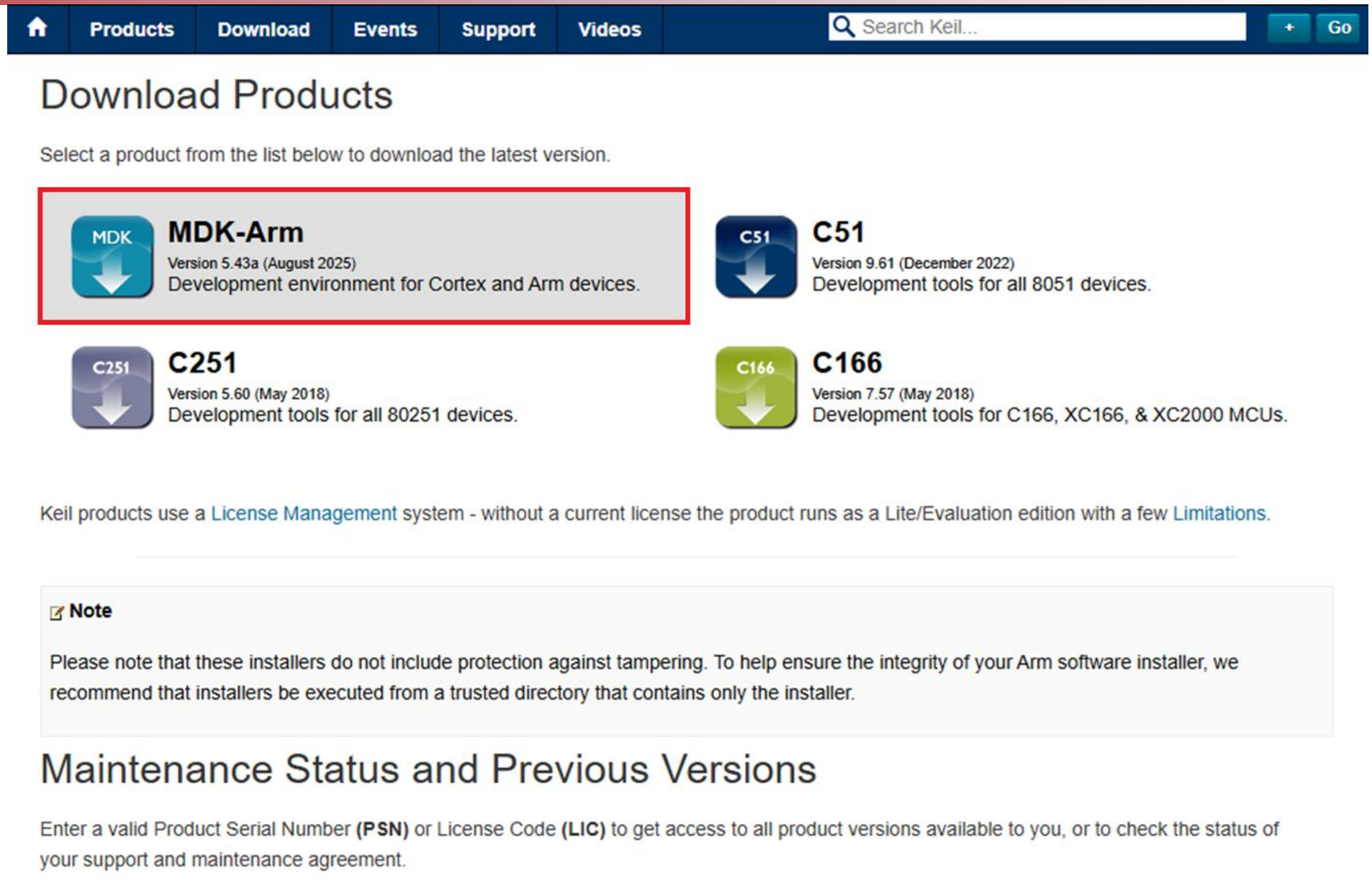
3.5. Keil/MDK-ARM Project Template



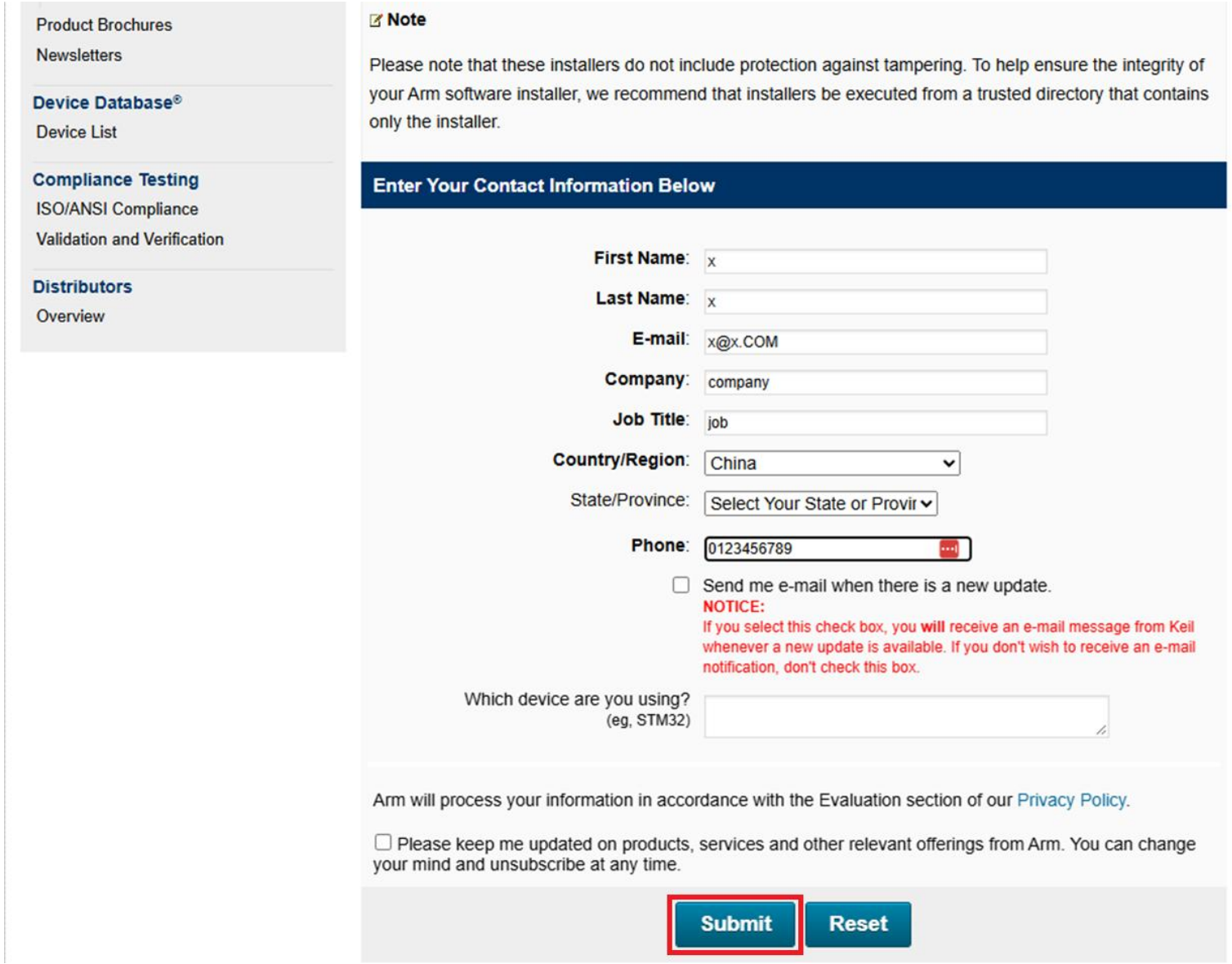
3.5. Keil/MDK-ARM Project Template



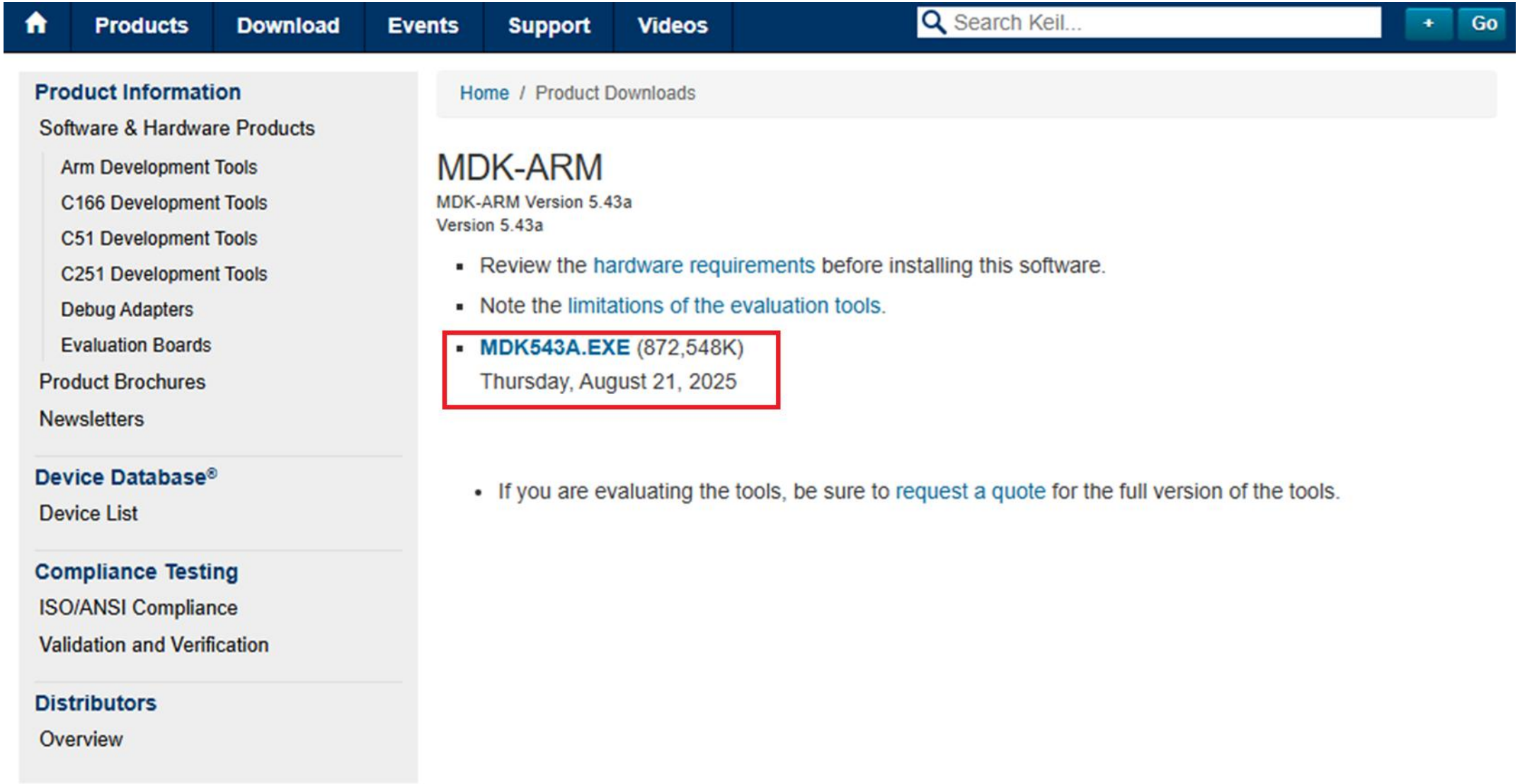
3.5. Keil/MDK-ARM Project Template



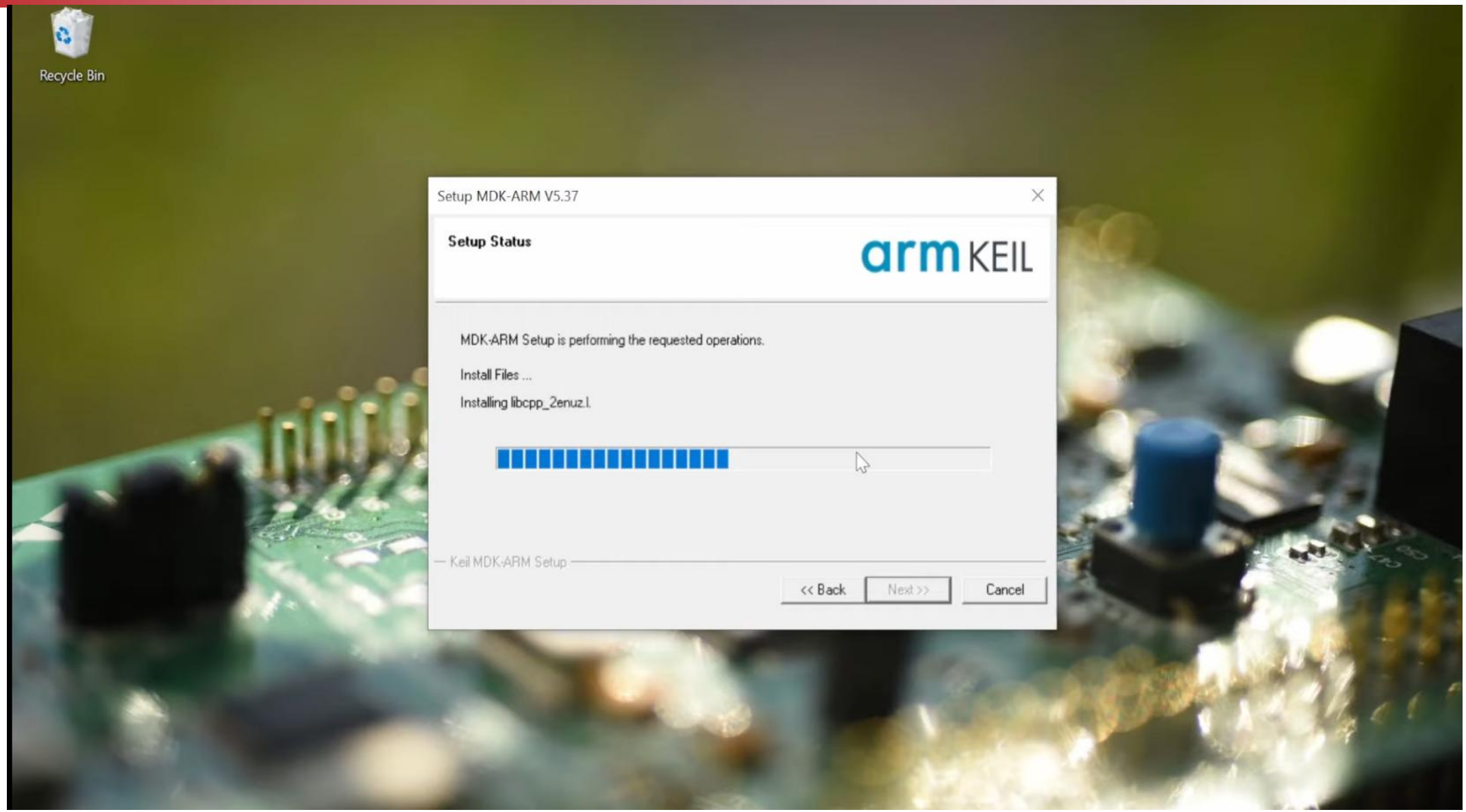
3.5. Keil/MDK-ARM Project Template



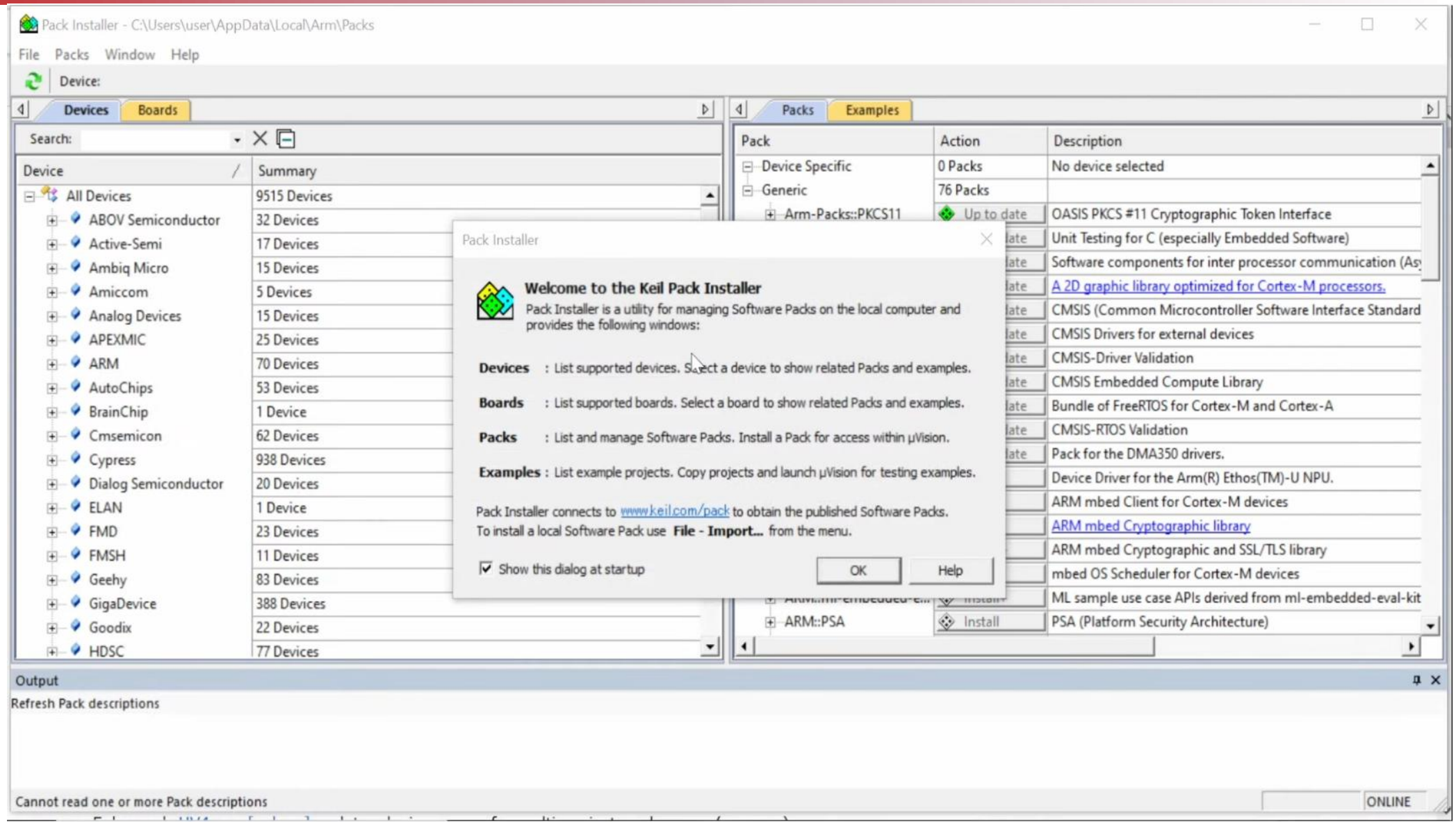
3.5. Keil/MDK-ARM Project Template



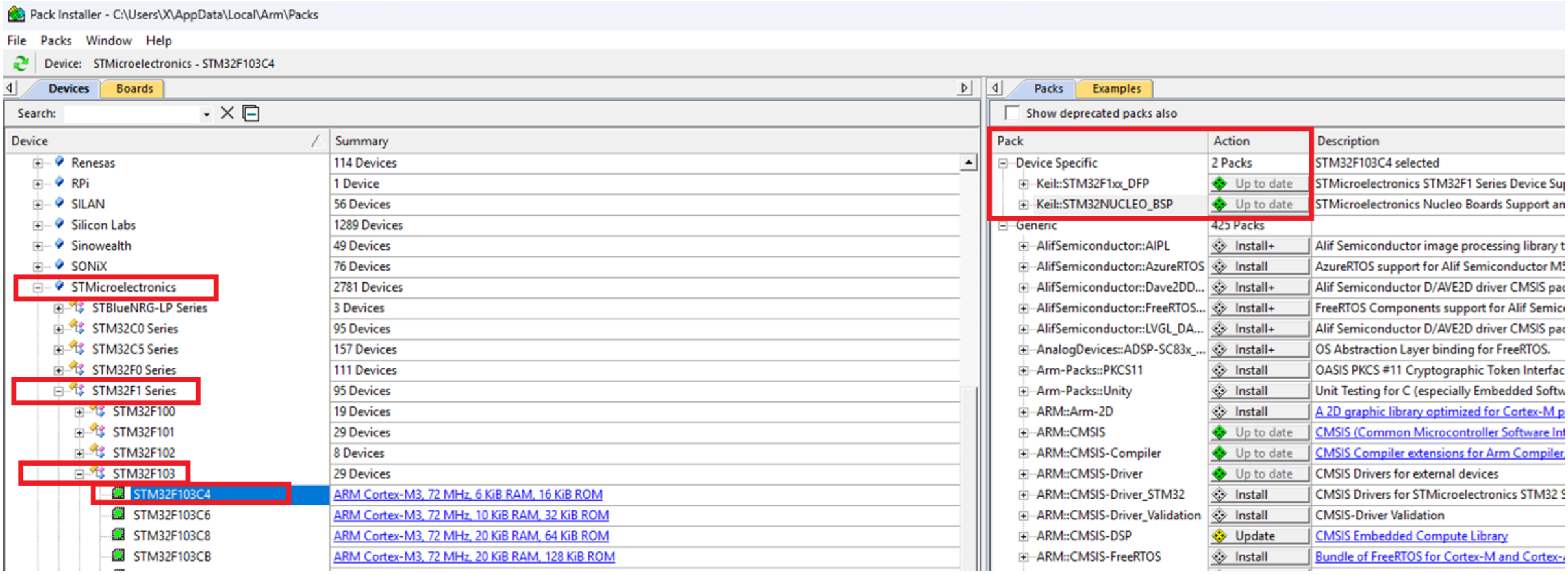
3.5. Keil/MDK-ARM Project Template



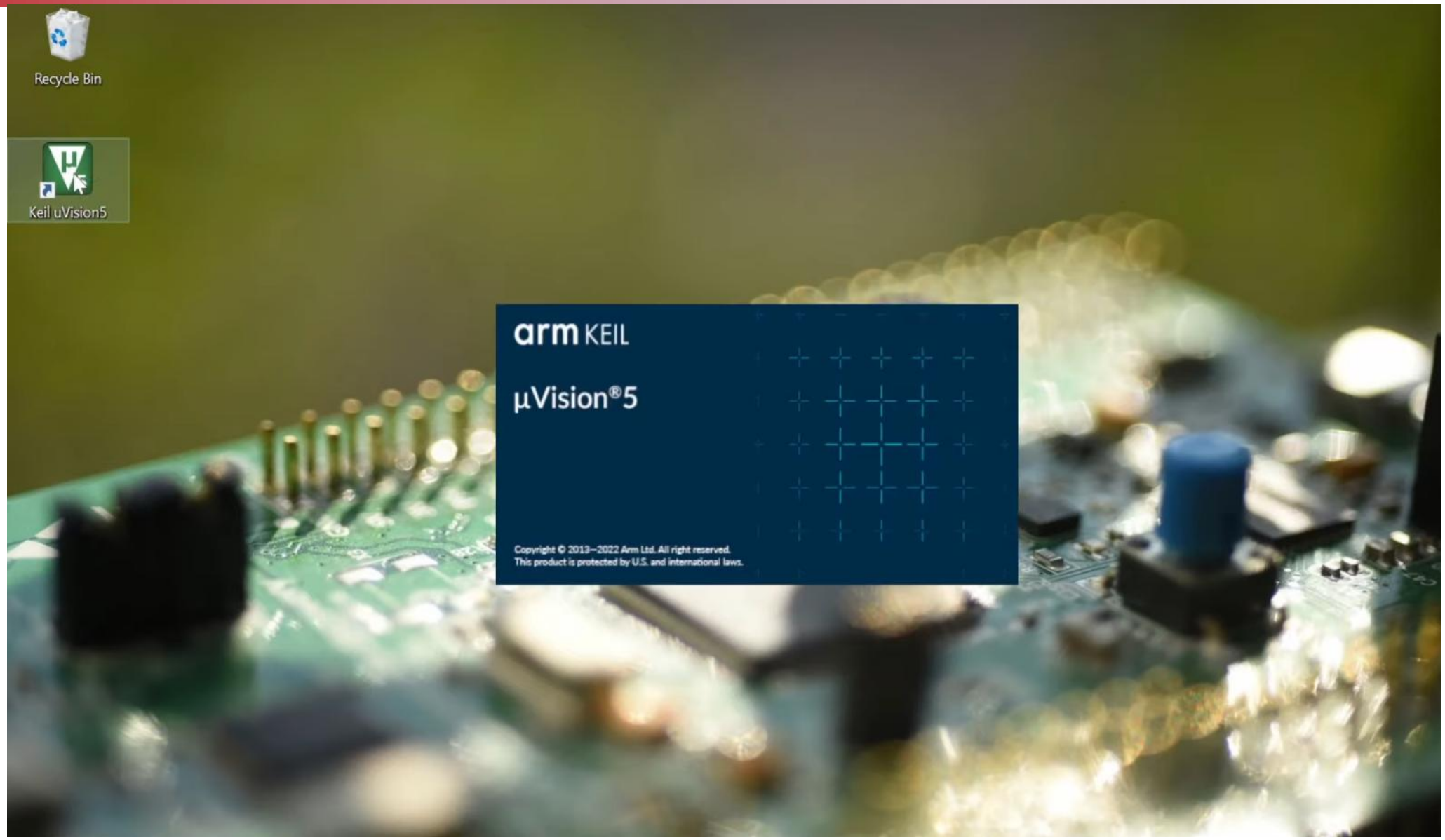
3.5. Keil/MDK-ARM Project Template



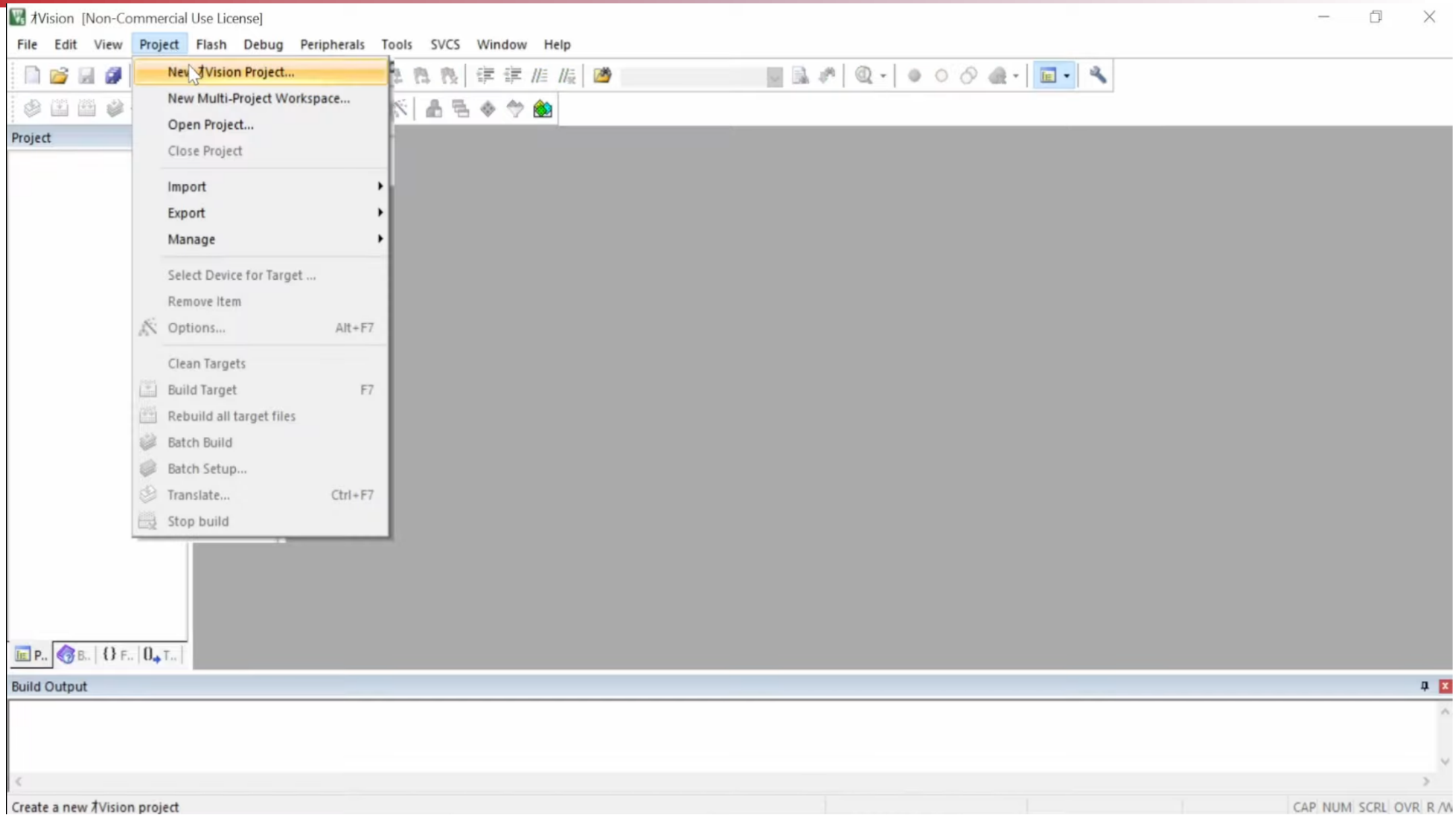
3.5. Keil/MDK-ARM Project Template



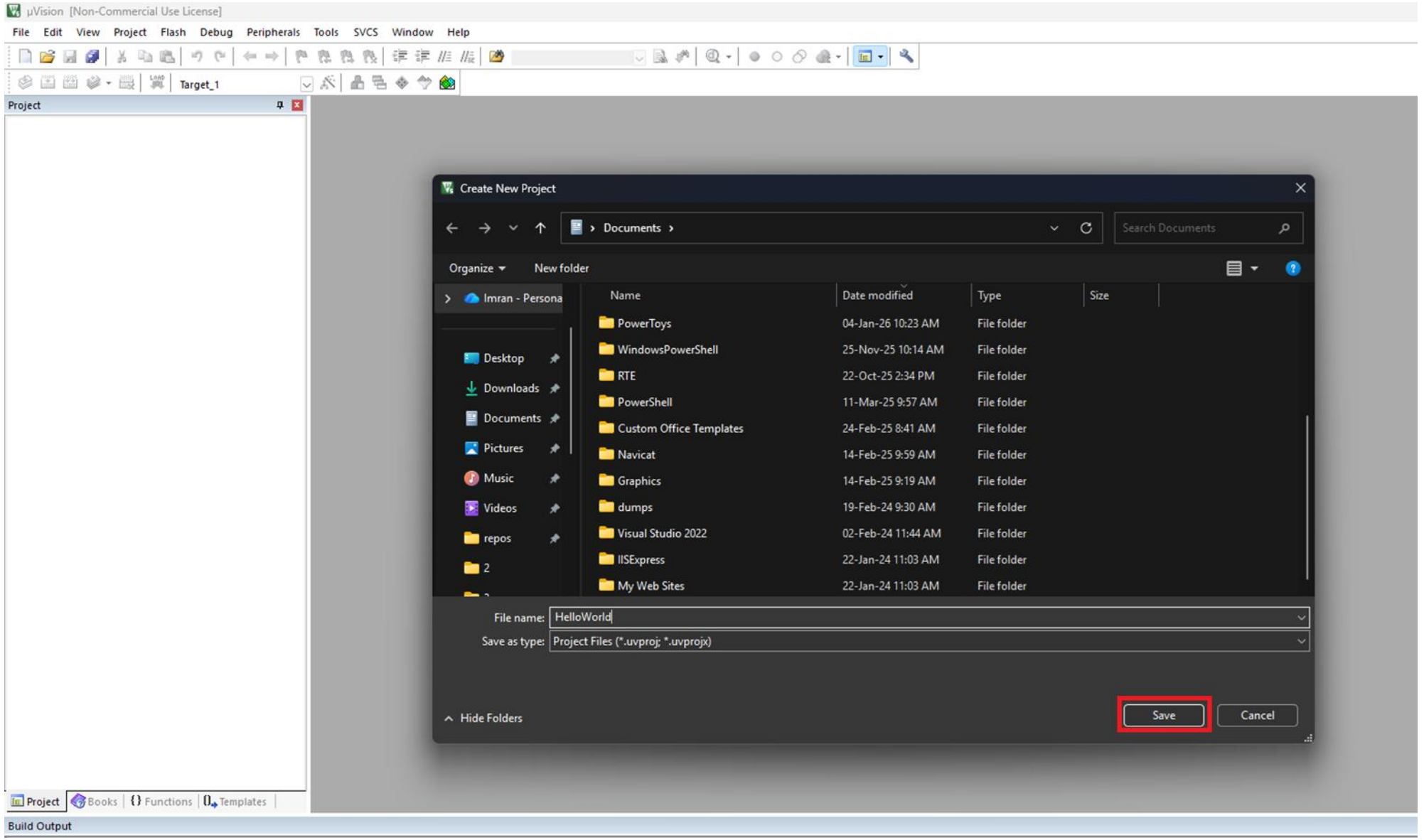
3.5. Keil/MDK-ARM Project Template



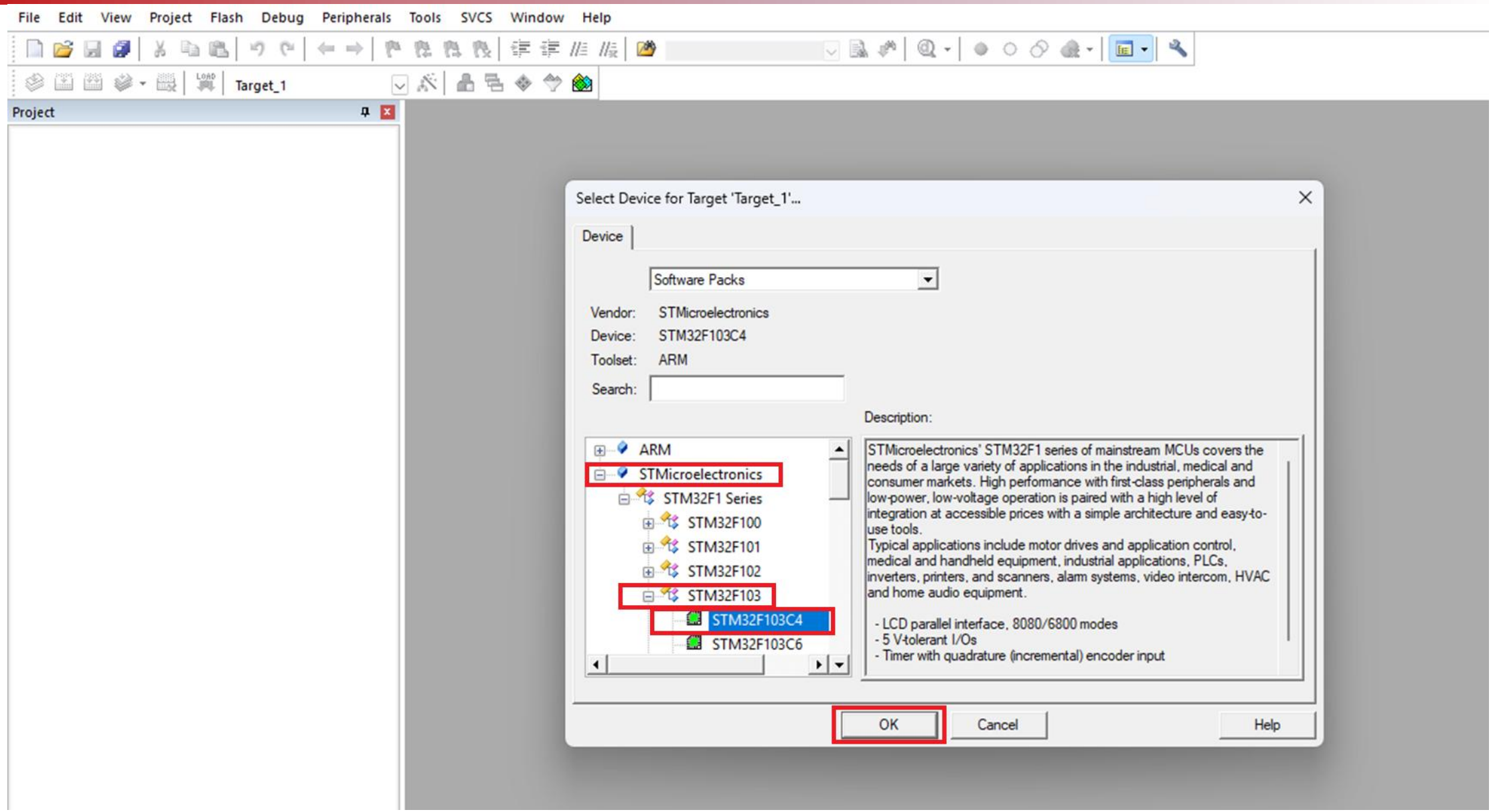
3.5. Keil/MDK-ARM Project Template



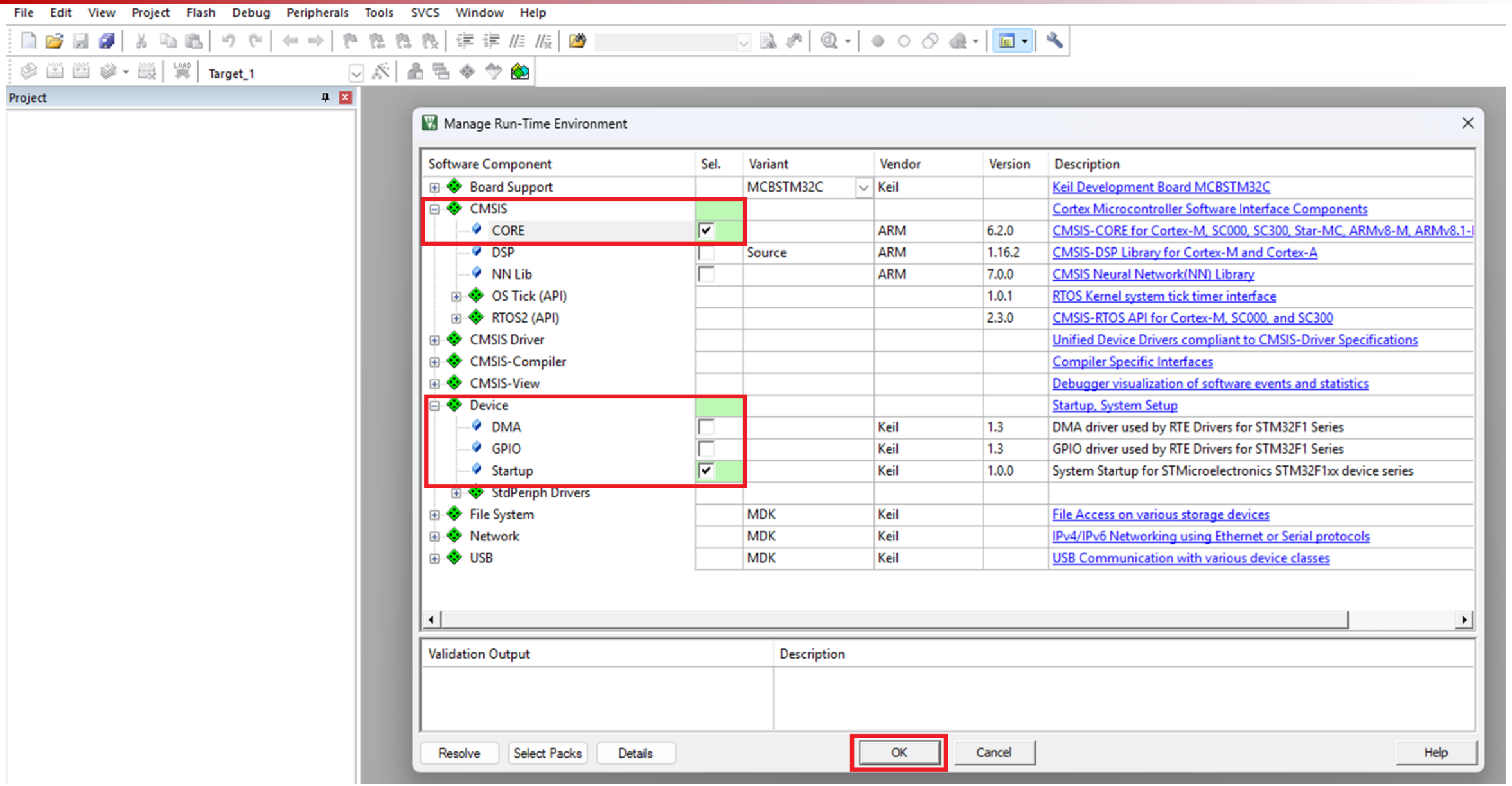
3.5. Keil/MDK-ARM Project Template



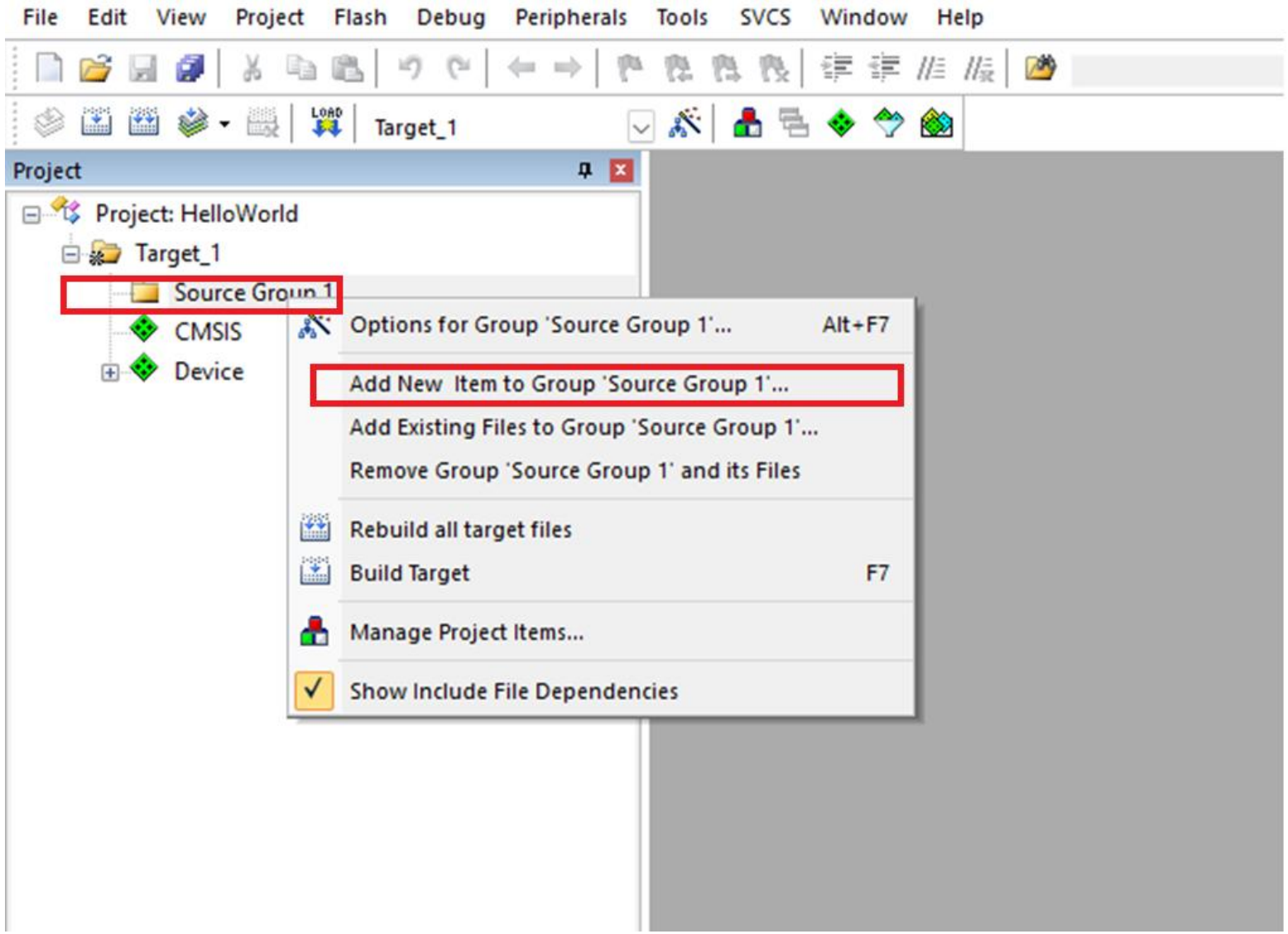
3.5. Keil/MDK-ARM Project Template



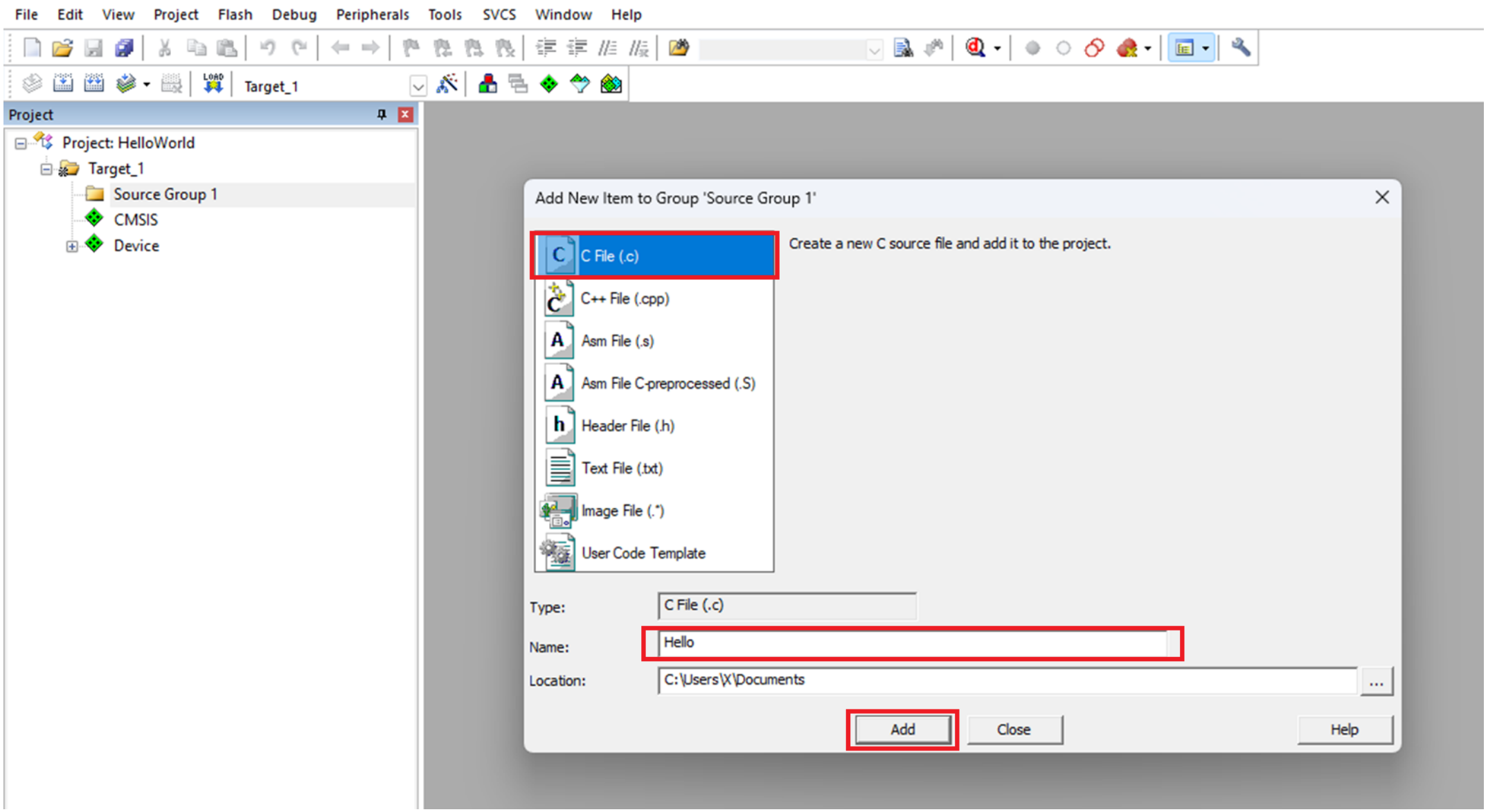
3.5. Keil/MDK-ARM Project Template



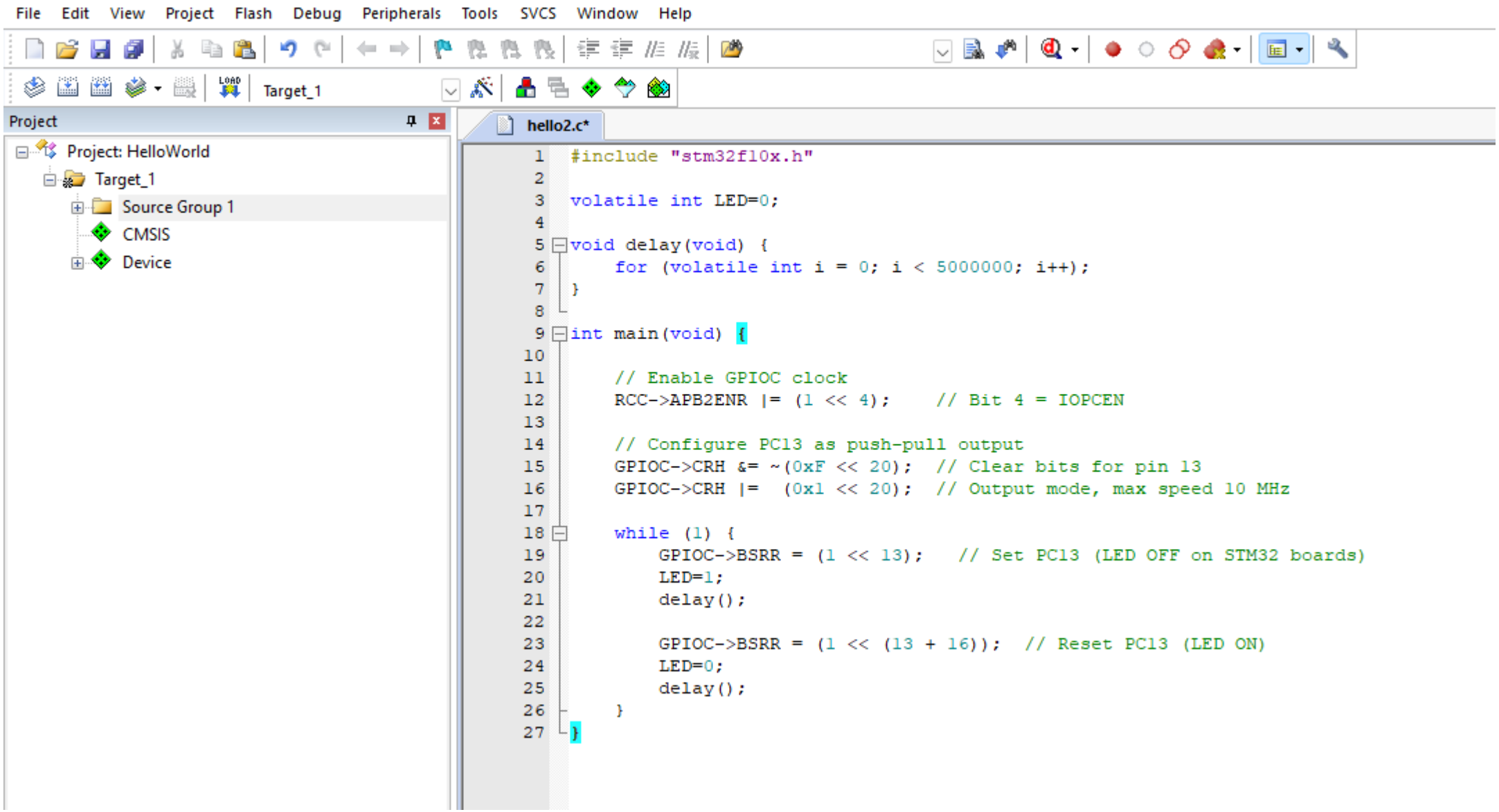
3.5. Keil/MDK-ARM Project Template



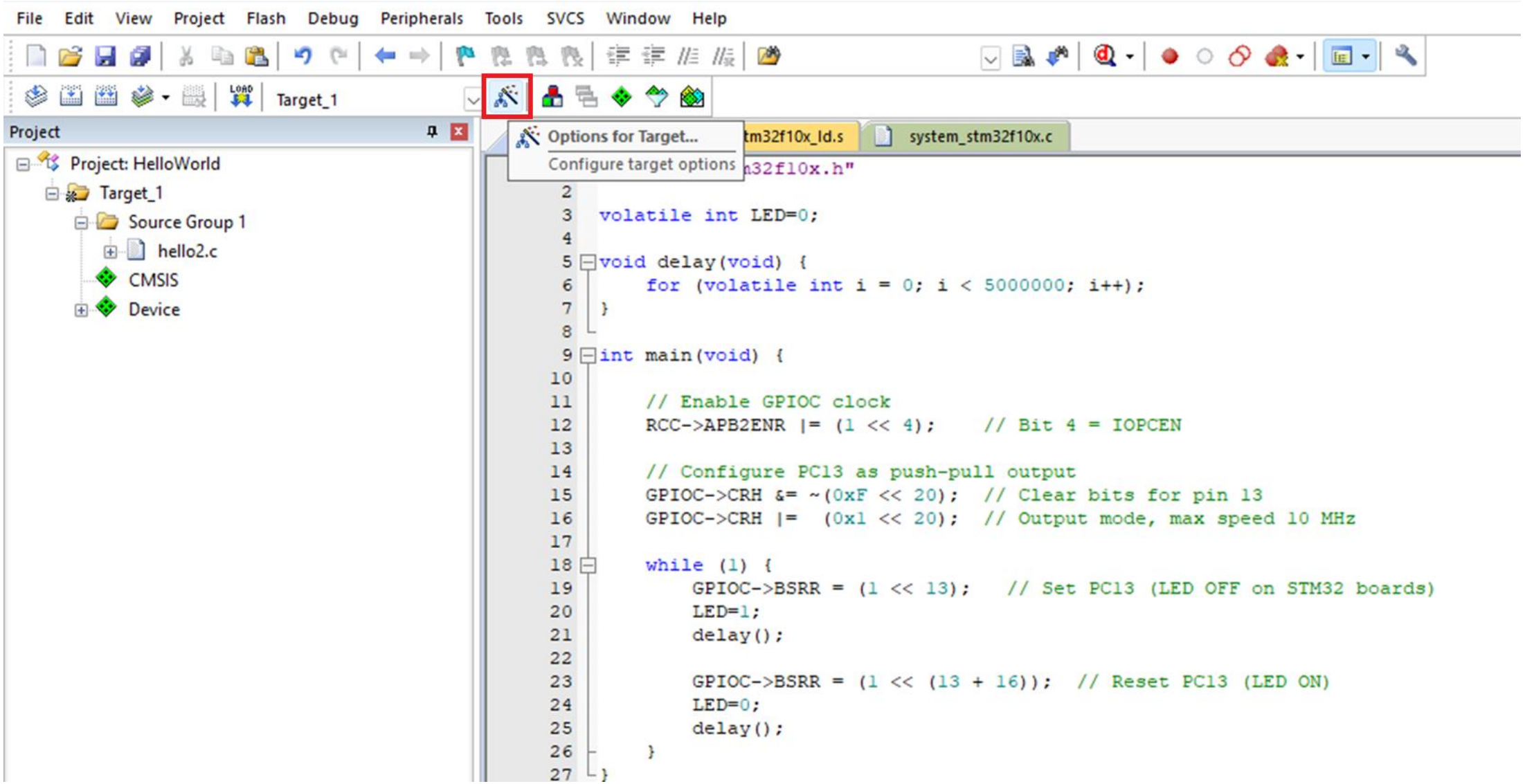
3.5. Keil/MDK-ARM Project Template



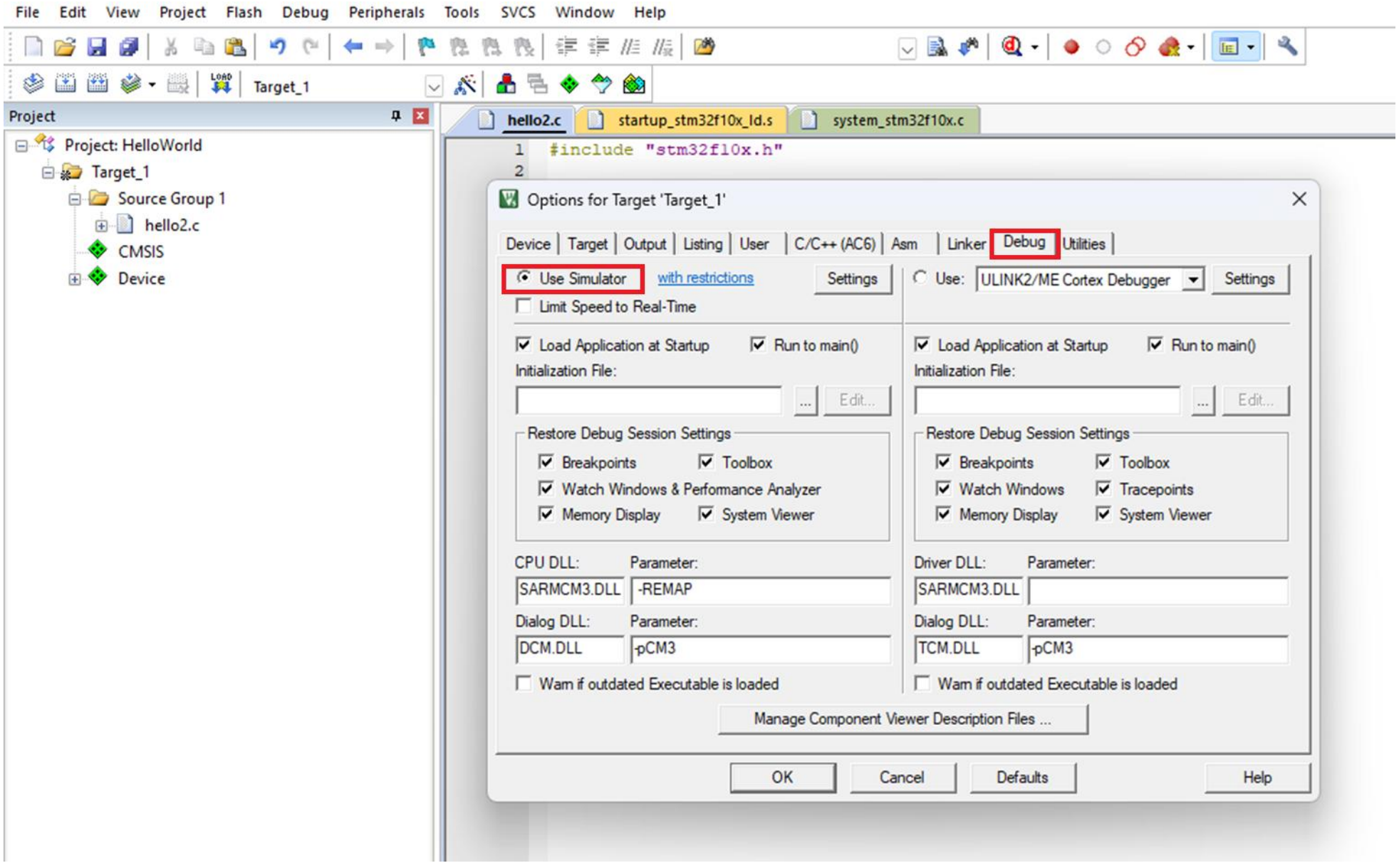
3.5. Keil/MDK-ARM Project Template



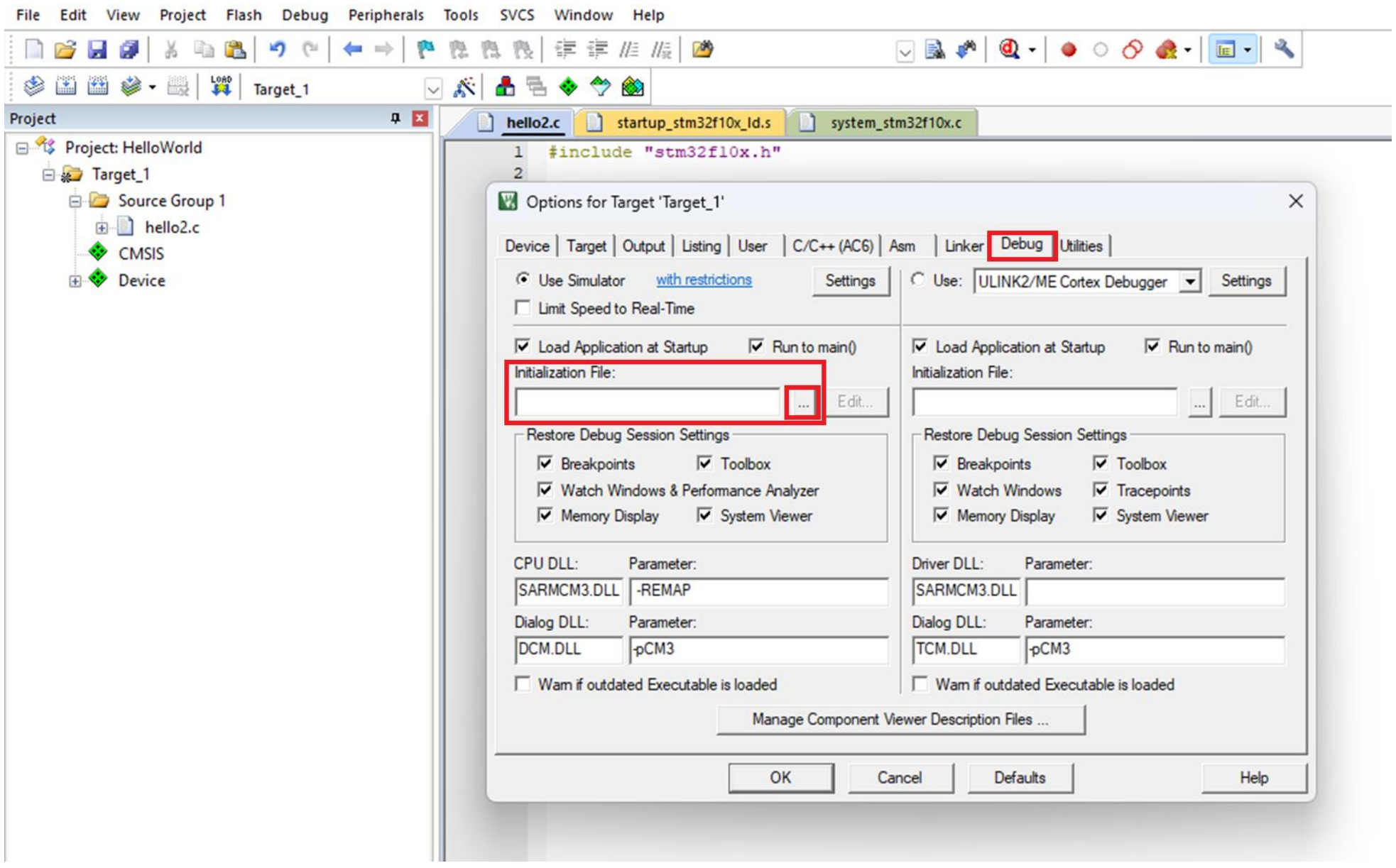
3.5. Keil/MDK-ARM Project Template



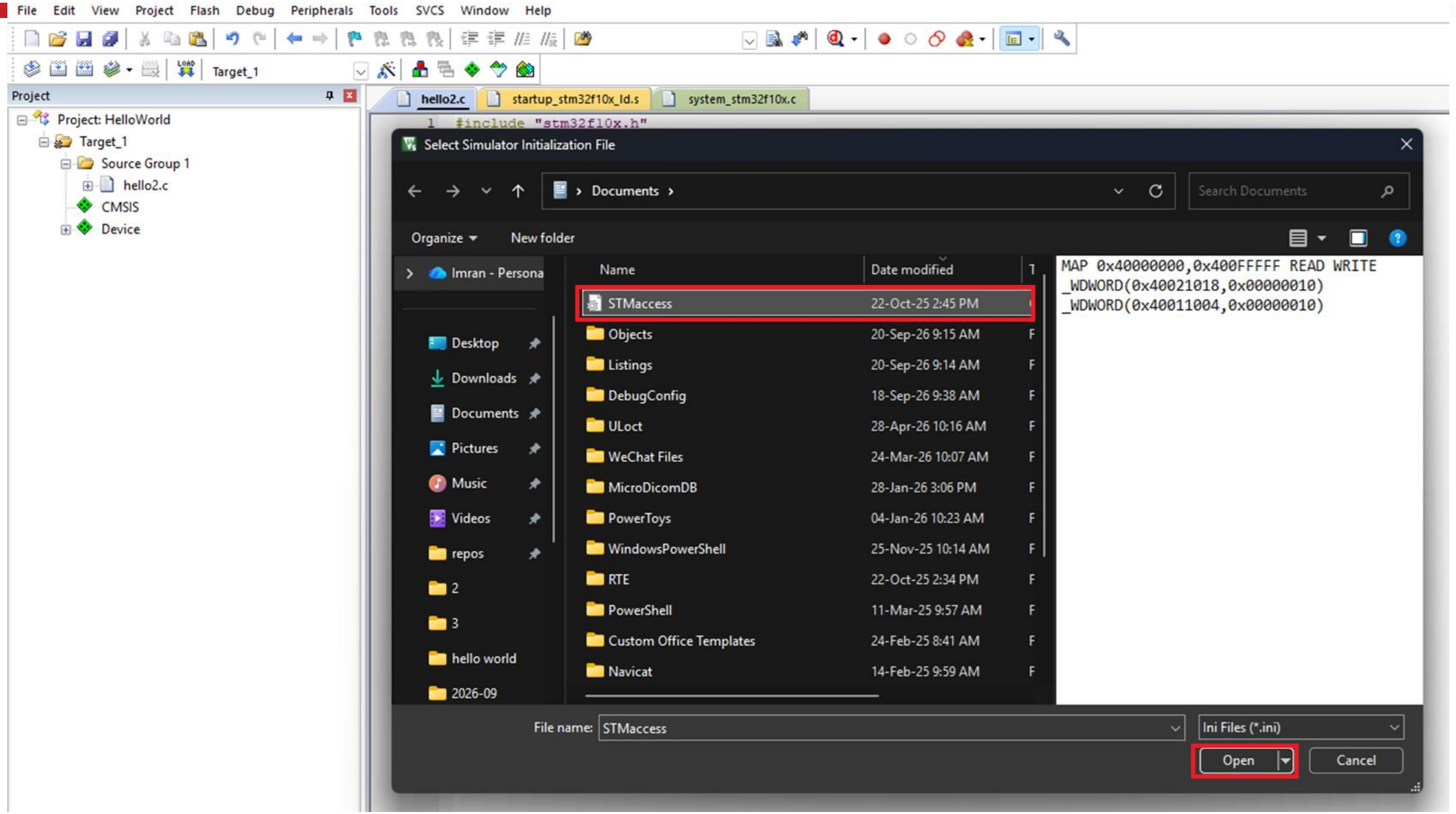
3.5. Keil/MDK-ARM Project Template



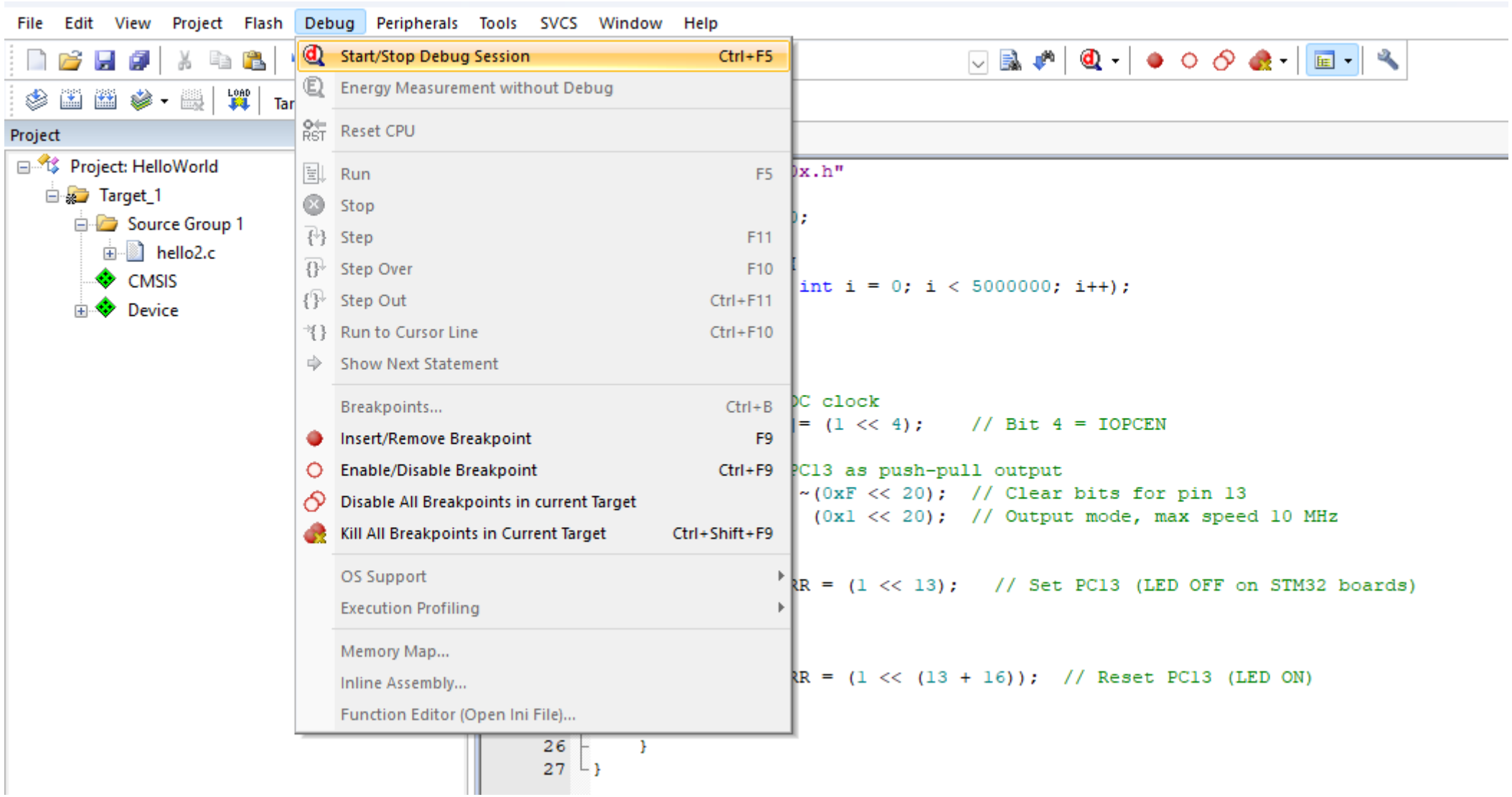
3.5. Keil/MDK-ARM Project Template



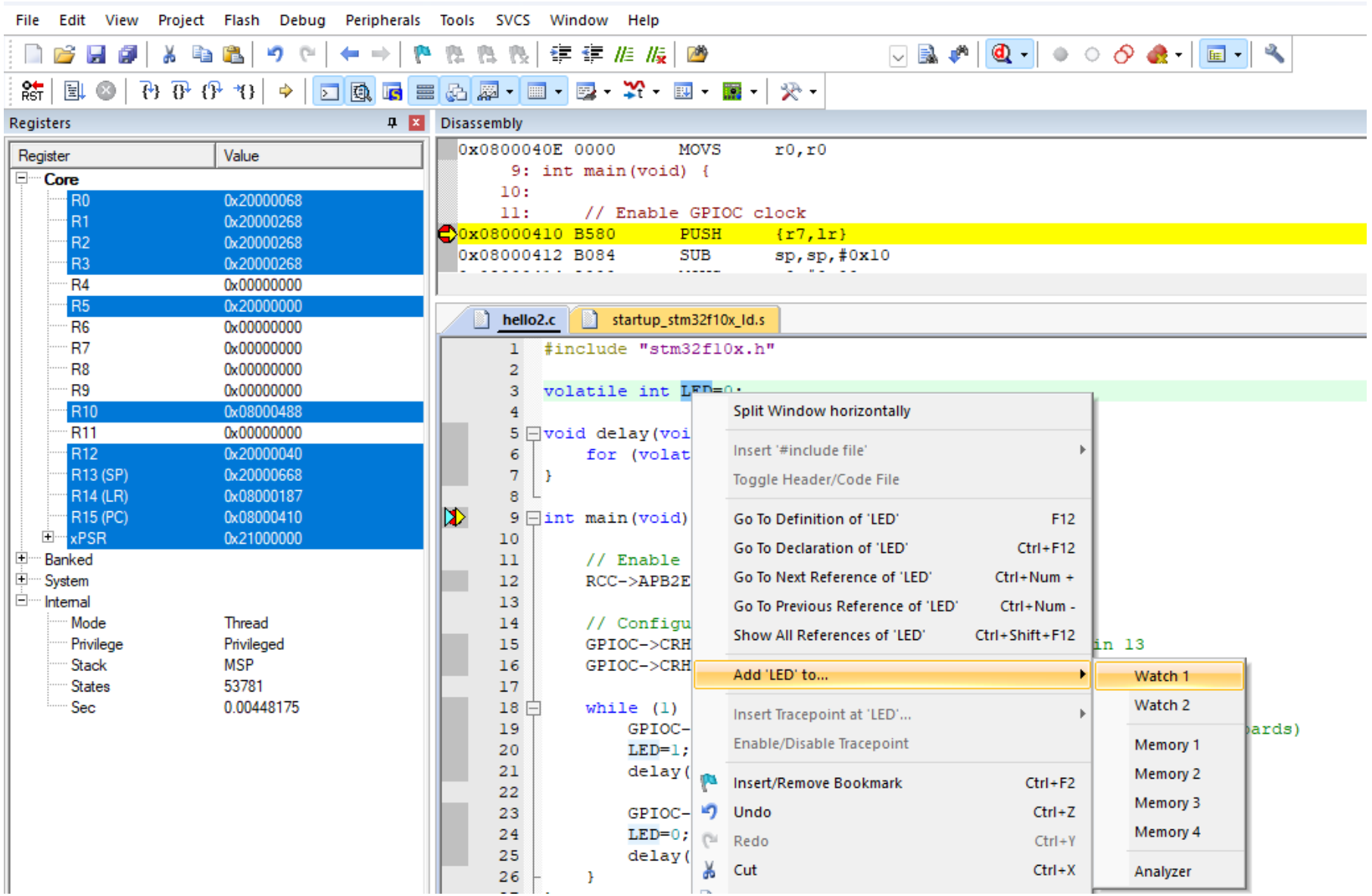
3.5. Keil/MDK-ARM Project Template



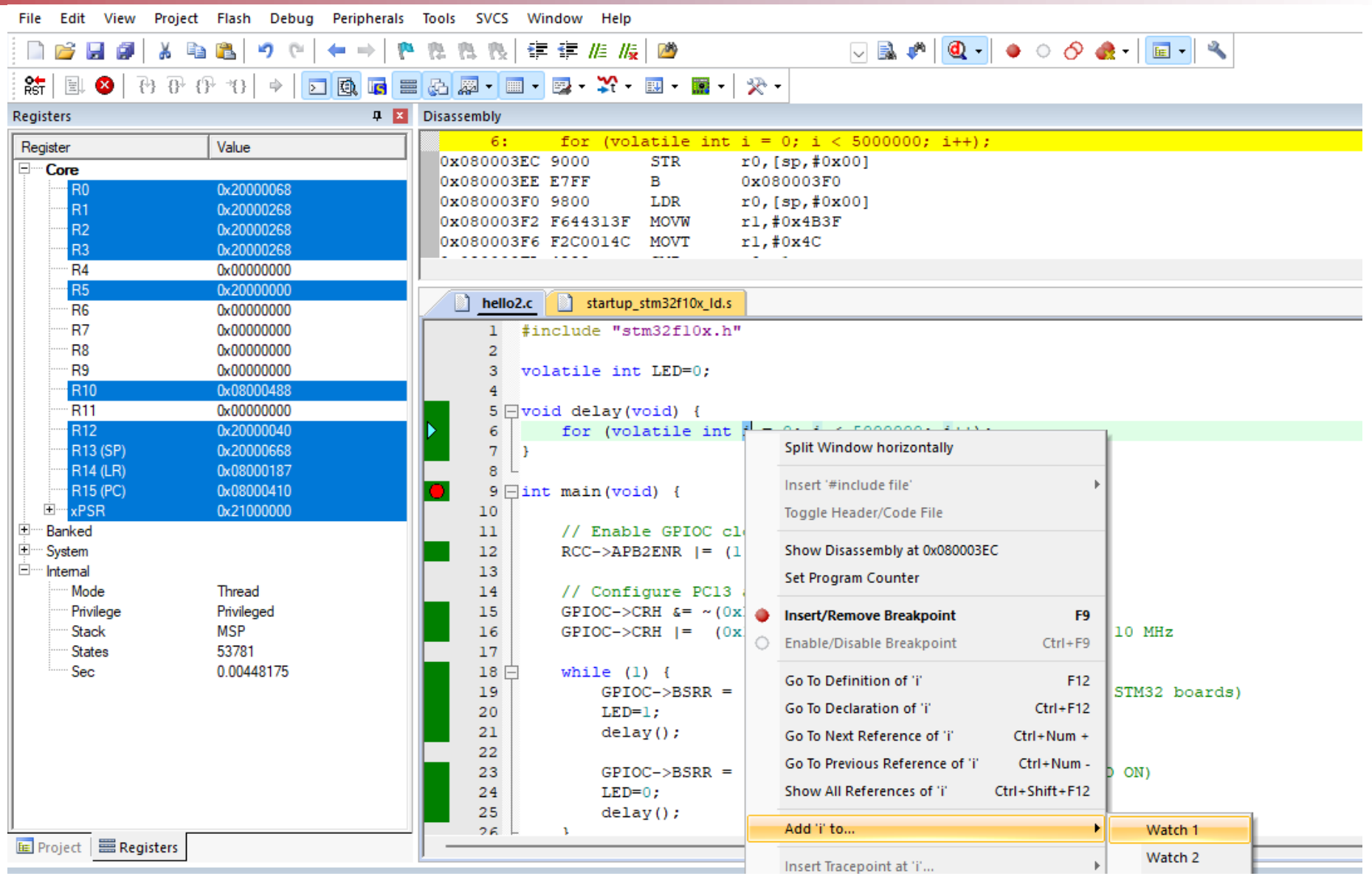
3.5. Keil/MDK-ARM Project Template



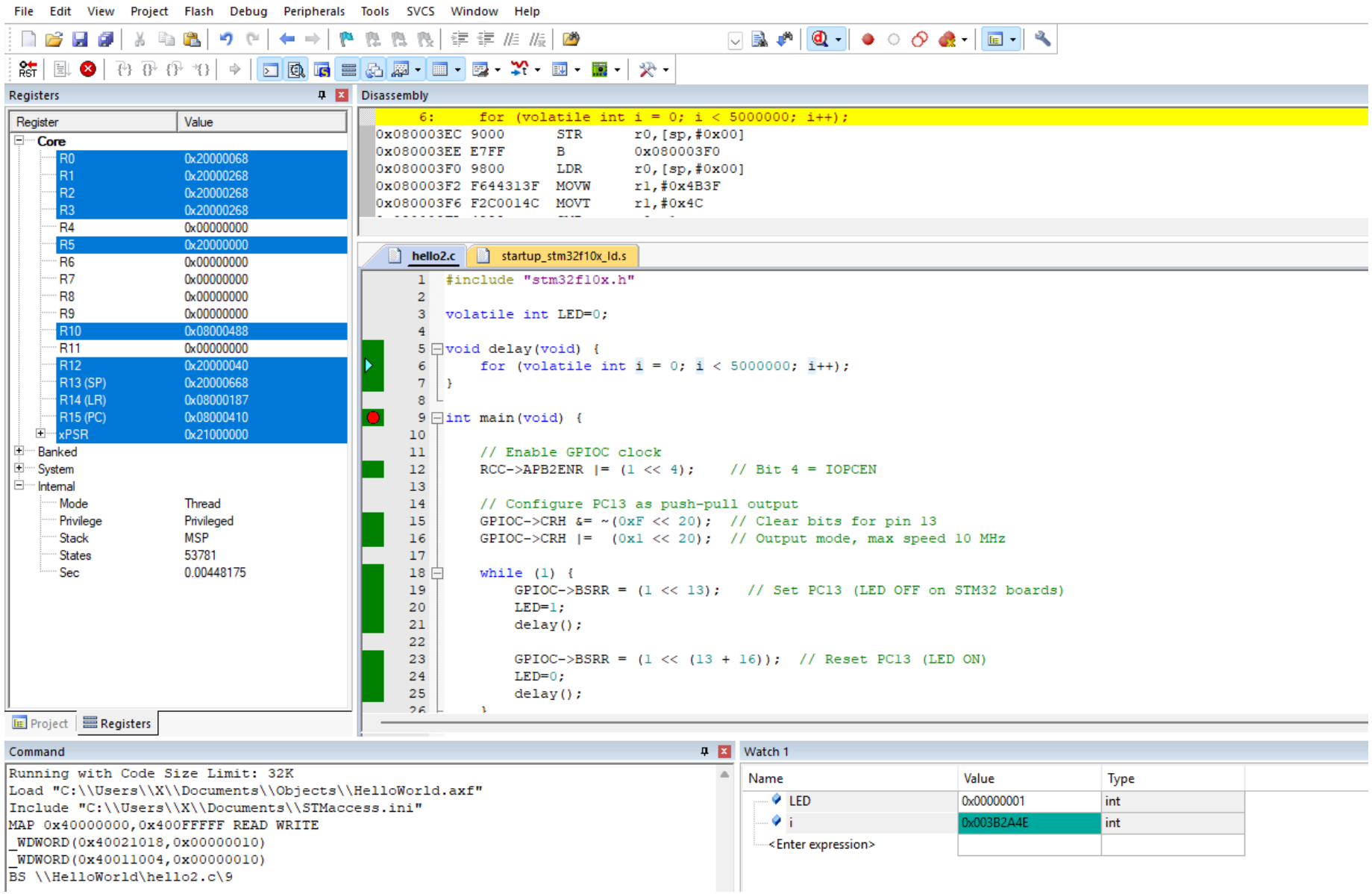
3.5. Keil/MDK-ARM Project Template



3.5. Keil/MDK-ARM Project Template



3.5. Keil/MDK-ARM Project Template



Contents

1. Introduction to GPIO
2. GPIO Features in STM32
   - Configurable Modes
   - Configurable Speed
   - Internal Pull-up/Pull-down Resistors
3. GPIO Registers in STM32
   - Important Registers
   - Steps to configure GPIO
4. Simulated Applications
   - Example 1: Flowing Light (Output Mode)
   - Example 2: Input Mode (Key Detection)
5. Keil/MDK-ARM Project Template
6. **Common Mistakes in Development**
7. Summary

3.6. Common Mistakes in Development

Forgetting to Enable GPIO Clock in RCC

- Before using any GPIO pin, its **clock must be enabled** in the RCC (Reset and Clock Control).
- If you forget this step, the pin won't respond, no matter how correct the rest of your code is.

- **Example:**

```
RCC->AHB1ENR |= (1 << 0);  // Enable clock for GPIOA
```

- **Analogy:** Like trying to use a TV without plugging it into power — it simply won't turn on.

3.6. Common Mistakes in Development

Wrong Pin Mode Configuration

- STM32 pins are **multiplexed** — the same pin can be input, output, analog, or alternate function.
- If you set the wrong mode in **MODER**, the pin won't behave as expected.

- **Example:** Setting LED pin as input instead of output → LED never turns on.

3.6. Common Mistakes in Development

Floating Input Pins (No Pull-up/Down)

- If an input pin is left "floating" (not tied to HIGH or LOW), it can pick up noise and change randomly.
- **Indication:** Button appears to press itself without touching it.
- **Solution:** Use **internal pull-up/pull-down resistors** via PUPDR register.

- **Example:**

```
GPIOA->PUPDR |= (1 << 0);  // Enable pull-up for PA0
```

3.6. Common Mistakes in Development

Incorrect Delays (Too Fast → No Visible LED Change)

- Microcontrollers run very fast (tens of MHz).
- If you blink an LED without adding delay, it will switch ON/OFF quickly for human eye to notice.
- **Indication:** LED looks like it never changes, even though it's toggling rapidly.
- **Solution:** Add software delays or timers.

- **Example:**

```
for (int i = 0; i < 100000; i++);  // Simple delay
```

Contents

1. Introduction to GPIO
2. GPIO Features in STM32
   - Configurable Modes
   - Configurable Speed
   - Internal Pull-up/Pull-down Resistors
3. GPIO Registers in STM32
   - Important Registers
   - Steps to configure GPIO
4. Simulated Applications
   - Example 1: Flowing Light (Output Mode)
   - Example 2: Input Mode (Key Detection)
5. Keil/MDK-ARM Project Template
6. Common Mistakes in Development
7. **Summary**

3.7. Summary

GPIO Basics

- General Purpose Input/Output pins are for the MCU to interact with the outside world.
- Each pin can be configured as Input, Output, Alternate Function, or Analog.

GPIO Registers

- MODER → Sets pin mode (Input, Output, Alt, Analog).
- ODR → Writes HIGH/LOW values to output pins.
- IDR → Reads the state of input pins.
- BSRR → Fast atomic set/reset of pins.
- PUPDR → Configures pull-up or pull-down resistors.

3.7. Summary

Configuration Steps

- Enable GPIO port clock in RCC.
- Configure pin mode in MODER.
- Set output type in OTYPER (Push-pull / Open-drain).
- Configure speed in OSPEEDR.
- Configure pull-up/pull-down in PUPDR.
- Write to ODR (for output) or read from IDR (for input).

Configurable Options

- **Input mode**: Floating, Pull-up, Pull-down.
- **Output mode**: Push-pull, Open-drain.
- **Speed**: Low (2 MHz), Medium (25 MHz), High (50 MHz), Very High (100 MHz).

3.7. Summary

Applications & Examples

- **Flowing Light (Output Example):** LEDs connected to GPIO pins turn ON one by one using bit-shifting on ODR.
- **Key Detection (Input Example):** Button connected to GPIO pin read through IDR; LED controlled based on input state.

Common Mistakes to Avoid

- Forgetting to enable GPIO clock in RCC.
- Wrong pin mode selection.
- Floating inputs without pull-up/down resistors.
- Missing delays in LED blink programs (too fast for the eye to see).

